DIFFUSION MODELS

From the Ink Drop to Robot Control

A Deep Learning Textbook

Forward & Reverse Processes · DDPM · Score-Based Models · Latent Diffusion · Diffusion Policy for Robotics

Prepared for: Sunkeerth

Engineering — Robotics, AI & Autonomous Systems

A continuation of Autoencoders and Variational Autoencoders: From First Principles to Practice

Table of Contents

Preface 4

PART I — Foundations: What Diffusion Actually Means 5

Chapter 1: The Physical Analogy — A Drop of Ink 5
Ink-drop/thermodynamics analogy, forward vs. reverse process, sense of real-world scale (T=1000)

Chapter 2: Connecting Diffusion to What You Already Know 8
Diffusion as a hierarchical VAE, diffusion as an Energy-Based Model, relationship to GANs/Autoregressive/Flows

Chapter 3: The Design Space — Variance-Preserving vs. Variance-Exploding 11
VP vs. VE forward-process formulations, why the distinction matters in practice

PART II — The Forward Process (Destruction) 13

Chapter 4: Adding Noise, One Step at a Time 13
The single-step Gaussian formula, Markov chain structure

Chapter 5: The Noise Schedule and the Closed-Form Shortcut 16
Linear vs. cosine schedules, the alpha-bar shortcut and its derivation

Chapter 6: Why Gaussian Noise, Specifically? 19
Closed-form composability, Central Limit Theorem intuition, Feller's reversibility result

Chapter 7: A Complete Worked Numerical Example 20
Hand-computed forward process on real numbers, verifying the shortcut formula

PART III — The Reverse Process (Creation): The Actual Learning Problem 22

Chapter 8: Why We Need a Neural Network At All 22
Intractability of the true reverse step, the exact closed-form posterior given x0

Chapter 9: What Should the Network Actually Predict? 24
Noise, x0, score, and velocity parameterizations and how they interconvert

Chapter 10: The U-Net — The Network That Does the Predicting 26
ResNet blocks, timestep conditioning, self/cross-attention, real model parameter counts

Chapter 11: Deriving the Training Objective, in Full 28
Full ELBO decomposition, Gaussian KL simplification, derivation of L_simple

Chapter 12: Full Training Algorithm and Production Code 31
Full PyTorch training loop, EMA weights, mixed-precision training

PART IV — Sampling: Turning Noise Into New Data 33

Chapter 13: The Sampling Loop 33
Step-by-step generation procedure and PyTorch sampling code

Chapter 14: Faster Sampling — DDIM and Beyond 35
DDIM's non-Markovian reformulation, deterministic sampling, DPM-Solver, distillation

Chapter 15: Controlling What Gets Generated 37
Classifier guidance, classifier-free guidance, negative prompts, guidance rescaling

PART V — The Modern Landscape 39

Chapter 16: Latent Diffusion Models (Stable Diffusion) 39
Compressing to latent space, perceptual+adversarial VAE loss, SD1.5 vs SDXL vs SD3

Chapter 17: The Score-Based / SDE Unification 41
Continuous-time SDEs, reverse-time SDE, Probability Flow ODE, score-noise equivalence

Chapter 18: Flow Matching 44
Straight-line conditional paths, velocity prediction, Diffusion Transformers

Chapter 19: Consistency Models — Toward One-Step Generation....46
Self-consistency property, consistency distillation vs. consistency training

Chapter 20: Evaluating Diffusion Models....47
FID, CLIP Score, precision/recall for generative models

PART VI — Diffusion Models for Robotics and Autonomous Systems....49

Chapter 21: Diffusion Policy — Robot Control as Denoising....49
Multimodal action generation, receding-horizon control, T_o/T_p/T_a, reported results

Chapter 22: World Models and Simulation via Diffusion....52
Predicted future frames, GAIA-1, Dreamer-style dynamics, synthetic training data

Chapter 23: Integrating a Diffusion Policy into ROS 2....53
Node architecture, QoS settings, TensorRT, a full latency budget

Chapter 24: Where to Go From Here....56
A concrete, ordered self-study path from here

PART VII — Exercises and Practice....57

Chapter 25: Exercises and Practice Problems....57
24 problems with full worked solutions across conceptual, computational, and design questions

Appendix A: Notation Reference....61

Appendix B: Glossary....62

Appendix C: Formula Cheat Sheet....63

Appendix D: Debugging Checklist for Diffusion Model Code....64

Appendix E: Index of Figures....65

Appendix F: Further Reading....66

Preface

You already know Transformers. You now also know Autoencoders and Variational Autoencoders in depth. This book takes the next step: **Diffusion Models** — the architecture behind Midjourney, Stable Diffusion, Sora, and (as you will see in the final part of this book) some of the most exciting recent work in robot control.

You shared a short visual guide covering the foundations of generative modeling — the height-guessing analogy for probability distributions, KL Divergence as a ruler for error, and the impossible Partition Function that blocks direct probability calculation. That guide ended by listing five families of generative models that all exist to dodge this one mathematical wall, and it closed with a single sentence pointing at where this book begins: **diffusion models combine the compression idea of VAEs with the energy-rolling idea of Energy-Based Models.** This book unpacks that sentence completely, from first principles to the current state of the art.

> **HOW TO READ THIS BOOK**
>
> Part I re-grounds the physical intuition (the ink-drop analogy) and connects it explicitly to what you already know about VAEs. Part II builds the forward (noising) process. Part III builds the reverse (denoising) process — the actual learning problem. Part IV covers how generation/sampling works once training is done. Part V covers the modern landscape (Latent Diffusion, score-based views, fast samplers). Part VI is where this all connects directly to robotics, autonomous systems, and ROS 2 — including Diffusion Policy, a technique generating robot actions the exact same way Stable Diffusion generates images.

PART I — Foundations: What Diffusion Actually Means

This part builds the intuition first, in plain language and pictures, then immediately grounds it in the exact machinery you already know from Autoencoders and Variational Autoencoders. By the end of Part I you will know precisely why diffusion models exist, what mathematical wall they were invented to get around, and where they sit relative to every other generative model family.

Chapter 1: The Physical Analogy — A Drop of Ink

1.1 Where the Name Comes From

Drop a single bead of ink into a glass of still water. At first, the ink is a tight, concentrated blob — a clear, structured shape. Over time, with no outside interference, the ink molecules bump around randomly (Brownian motion) and spread out. Eventually the ink is uniformly mixed through the entire glass: no structure left, just an even, undifferentiated haze.

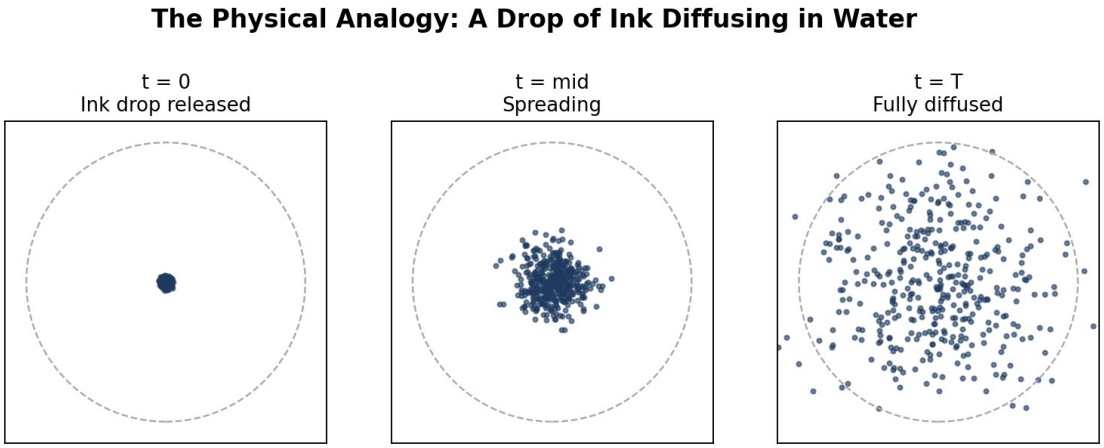


Figure 1.1 — A structured drop of ink gradually loses all of its structure as it diffuses through water. This physical process gives diffusion models their name.

This physical process — concentrated structure slowly collapsing into featureless randomness — is a real phenomenon studied in thermodynamics called **diffusion**. It is formally described by the Fokker-Planck equation, which tracks how a probability density spreads out over time under random molecular motion. A Diffusion Model borrows this exact mathematical description and runs it, conceptually, in reverse — on data instead of ink.

1.2 The Two Halves of the Idea

A diffusion model is built entirely from two processes, and everything in this book is really just precision-engineering these two halves:

- **Forward process (destruction)** — take a real image, and, step by step, add a little random noise until nothing recognizable is left — exactly like the ink spreading into water. This half requires **no learning at all**; it is simple, fixed, known mathematics (Part II).
- **Reverse process (creation)** — train a neural network to undo this, step by step, turning pure random noise back into a structured, realistic image — like watching the ink video in reverse, and the ink spontaneously reassembling itself back into a tight drop. This half is the **entire learning problem** (Part III).

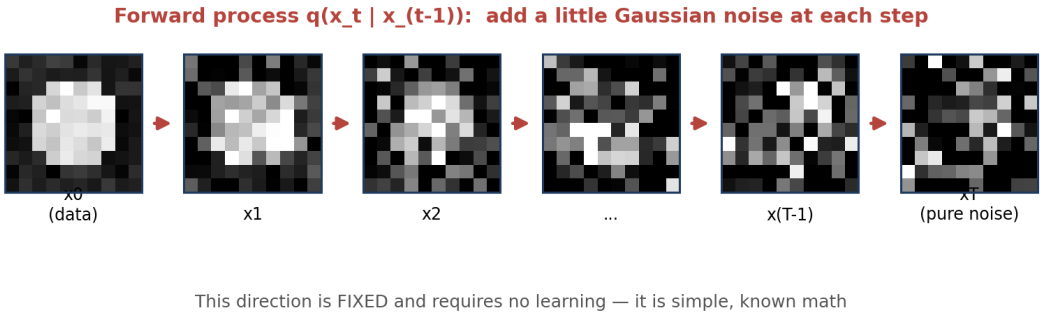


Figure 1.2 — The forward process: real data, step by step, becomes indistinguishable noise.

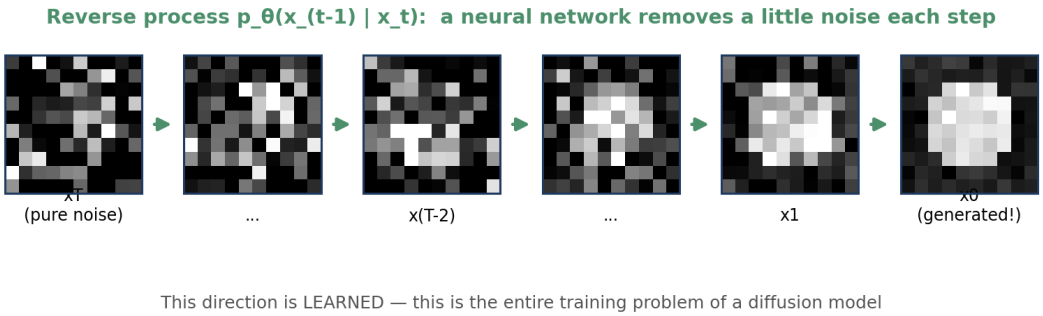


Figure 1.3 — The reverse process: a trained network turns pure noise back into a plausible sample, one small denoising step at a time.

> **KEY IDEA**
>
> If ink spreading into water is like forgetting, generating an image with a diffusion model is like carefully, deliberately remembering — rebuilding structure out of nothing, one small, confident step at a time.

1.3 A Sense of Scale

In a production system like Stable Diffusion, T is typically 1000 for training. A single forward step changes the image by only a tiny amount — imperceptible to the eye. It takes hundreds of these tiny steps stacked together before the image is unrecognizable. This "many tiny steps" design is not an accident: it is precisely what makes

each individual reverse step easy enough for a neural network to learn reliably, which you will see formalized in Chapter 5 and again in Chapter 11.

Chapter 2: Connecting Diffusion to What You Already Know

2.1 Diffusion Models Are (Almost) Hierarchical VAEs

Recall from your Autoencoders and Variational Autoencoders study: a VAE has one encoder step ($x \rightarrow z$) and one decoder step ($z \rightarrow x$), regularized by a KL divergence term so the latent space is smooth and samplable. A **Diffusion Model is exactly this idea, stretched out across hundreds or thousands of tiny steps instead of one big jump.**

VAE concept	Diffusion model equivalent
Encoder $q(z\|x)$	Forward process $q(x_t \mid x_{t-1})$ — fixed, not learned
Decoder $p_\theta(x\|z)$	Reverse process $p_\theta(x_{t-1} \mid x_t)$ — learned
Single latent code z	A whole chain of latents x1, x2, ..., xT
Prior $p(z) = N(0,1)$	Prior $p(x_T) = N(0,1)$ — the fully-noised end state
ELBO loss (reconstruction + KL)	A sum of many small per-step KL terms (Chapter 11)

This is not just a loose metaphor — it is mathematically precise. A diffusion model's training objective is derived using the **exact same ELBO machinery** you already learned for VAEs (Chapter 11 walks through this explicitly, including the closed-form Gaussian posterior). If you understood the VAE's ELBO derivation, you are most of the way to understanding a diffusion model's loss function already.

2.2 Diffusion Models Are Also Energy-Based Models

Your uploaded guide described Energy-Based Models (EBMs) using a valley analogy: realistic data sits at the bottom of a low-energy valley, and generating new samples means rolling random points downhill until they settle at the bottom. A diffusion model's reverse process does something extremely similar — at every step, it nudges a noisy point toward the region where real data is more probable, which is mathematically the same as following the *gradient* of a probability landscape (formalized as the **score function** in Chapter 17).

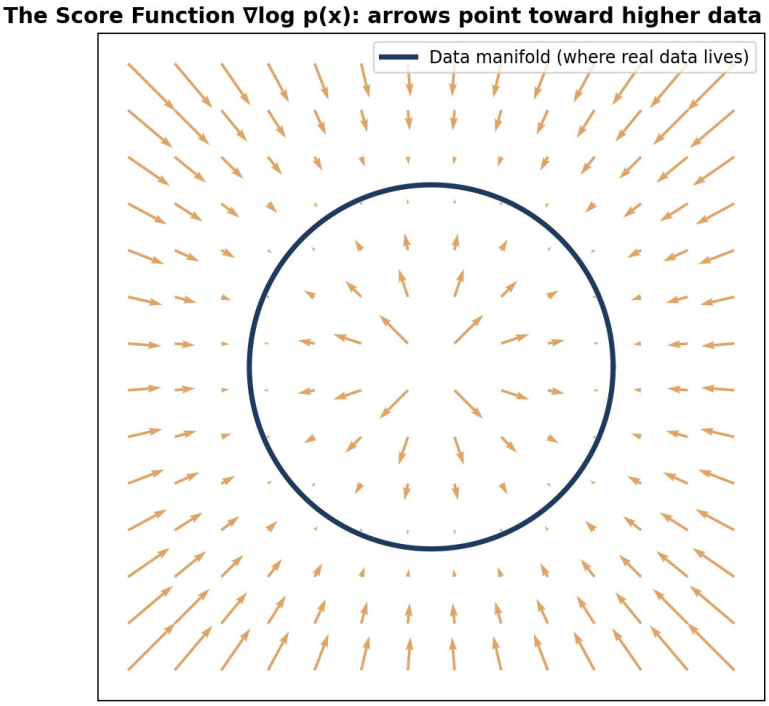


Figure 2.1 — The score function points every location in space toward the nearest region of high data density — the same "roll downhill" intuition as an Energy-Based Model.

2.3 Where This Leaves the Other Three Families

Your uploaded guide named five families: GANs, Autoregressive models, VAEs, Energy-Based Models, and (implicitly) Diffusion. It is worth being precise about diffusion's relationship to each of them:

Family	Relationship to Diffusion
GAN	Unrelated mechanism (adversarial game); diffusion uses no discriminator at all
Autoregressive	Unrelated mechanism (token-by-token); though VQ-VAE + Transformer pipelines share some spirit
VAE	Direct ancestor — diffusion is a hierarchical VAE with a specific, fixed encoder family
Energy-Based Model	Direct ancestor — the score function is precisely an EBM's gradient, learned implicitly
Normalizing Flow	A cousin — Flow Matching (Chapter 18) merges flow-based ideas directly into diffusion

WHY THIS MATTERS

A Diffusion Model is not really a sixth, unrelated family of generative model. It is what you get when you combine the VAE's step-by-step latent-chain structure with the EBM's gradient-following intuition, and apply both across many small steps instead of one large one. This is precisely why diffusion models

> sidestep the impossible Partition Function your uploaded guide described: instead of computing one impossibly complex distribution in one shot, the problem is broken into hundreds of small, individually tractable Gaussian sub-problems.

Chapter 3: The Design Space — Variance-Preserving vs. Variance-Exploding

Once you understand the basic forward/reverse idea, a natural engineering question arises: exactly how should noise be added? This chapter previews a distinction that becomes important again in Chapter 17, and is exactly the kind of detail that separates a surface-level understanding from a production-grade one.

3.1 Variance-Preserving (VP) — What This Book Uses

The formula introduced in Part II, $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1-\bar{\alpha}_t)} \cdot \varepsilon$, is called **variance-preserving**: notice that if x_0 has unit variance, x_t also has unit variance at every timestep, because the two coefficients under the square roots always sum to 1. The signal shrinks and noise grows, but the total "energy" stays constant throughout. This is the formulation used by DDPM and by Stable Diffusion.

3.2 Variance-Exploding (VE) — The Alternative

A different, equally valid design instead keeps the original signal at full strength and simply adds increasing amounts of noise on top, without shrinking the signal at all:

```
x_t = x_0 + σ_t · ε,      where σ_t grows from ≈ 0 up to a large value
```

READING THIS FORMULA, TERM BY TERM

- **`x_0`** — the original clean data — notice it appears with a coefficient of exactly 1, i.e. it is NEVER shrunk, unlike the VP formula
- **`σ_t`** — (sigma sub t) the noise scale at step t — starts near 0 and grows large, rather than being paired with a shrinking signal coefficient
- **`σ_t · ε`** — as t grows, this noise term keeps growing in absolute size, with nothing shrinking to balance it out — hence "exploding" variance

Here the total variance of x_t *explodes* as t grows, since nothing shrinks the original signal. This formulation is used by Score SDE / NCSN-style models (Chapter 17). It turns out, remarkably, that VP and VE are just two different parameterizations of the same underlying continuous-time process — a fact made precise by the unifying SDE framework you will see in Chapter 17.

3.3 Why This Detail Matters in Practice

This choice affects numerical stability, how the noise schedule should be tuned, and which sampler formulas apply. If you ever read a diffusion paper's code and wonder why one repository's noise schedule looks completely different in scale from another's, this VP/VE distinction is very often the reason — knowing it exists will save you real debugging time.

PART II — The Forward Process (Destruction)

This part makes the forward process fully precise: the exact formula for a single step, the shortcut that lets us skip directly to any timestep, why the noise must be Gaussian, and a hand-worked numerical example so you can verify every formula yourself. Every formula below is followed by a plain-English, symbol-by-symbol breakdown — read the formula, then read the box underneath it before moving on.

Chapter 4: Adding Noise, One Step at a Time

4.1 A Single Forward Step

The forward process defines how to go from x_(t-1) to a slightly noisier x_t. At every step, we shrink the previous image slightly toward zero and add a small amount of fresh Gaussian noise:

```
x_t = √α_t · x_(t-1)  +  √(1-α_t) · ε,     ε ~ N(0,1)
```

READING THIS FORMULA, TERM BY TERM

- **`x_t`** — the image AFTER this noising step — the thing we're computing
- **`x_(t-1)`** — the image BEFORE this step, one step less noisy
- **`α_t`** — (“alpha sub t”) a fixed number just under 1, e.g. 0.9998, chosen in advance — not learned. It says how much of the previous image to keep at this specific step
- **`√α_t`** — the SHRINK factor. Since α_t is a little less than 1, its square root is also a little less than 1, so multiplying x_(t-1) by it pulls the image slightly toward zero
- **`√(1-α_t)`** — the NOISE STRENGTH for this step — how loudly the random term ε gets mixed in. Bigger when α_t is smaller (more noise added)
- **`ε`** — (“epsilon”) one fresh random draw of noise, re-sampled independently at every step
- **`ε ~ N(0,1)`** — read the tilde as “is drawn from.” This says ε comes from the standard Gaussian (bell curve) with mean 0 and variance 1 — centered at zero, no systematic bias in any direction

Here α_t (alpha_t) is a number just under 1 (e.g. 0.9998) that controls how much of the previous image survives this particular step. Notice this has exactly the same shape as the reparameterization trick from Chapter 10 of the VAE book — a deterministic combination of a signal term and a noise term.

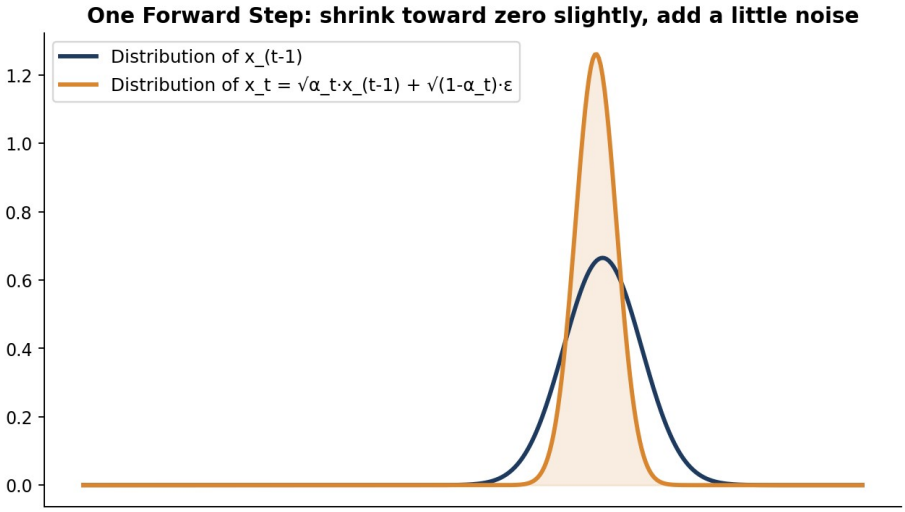


Figure 4.1 — One forward step shifts and widens the distribution slightly — a small, controlled loss of information.

4.2 Why This Specific Formula?

This equation defines a conditional Gaussian distribution:

```
q(x_t | x_(t-1)) = N( x_t ;  √α_t · x_(t-1),   (1-α_t)·I )
```

READING THIS FORMULA, TERM BY TERM

`q( · )` — the name we give to the (fixed, known) forward-process distribution — not something we're learning

`x_t | x_(t-1)` — read the vertical bar as "given." This is "the distribution of x_t, GIVEN we already know x_(t-1)"

`N( x_t ; mean, variance )` — "x_t follows a Normal/Gaussian distribution with this mean and this variance" — the semicolon just separates the variable from its two parameters

`√α_t · x_(t-1)` — the MEAN of that Gaussian — i.e. the most likely, average value of x_t is a shrunk copy of x_(t-1)

`(1-α_t)·I` — the VARIANCE — how spread out x_t can land around that mean. I is the identity matrix, meaning each dimension/pixel gets this same amount of independent noise

The mean is a shrunk version of the previous step (√α_t × x_(t-1)), and the variance (1-α_t) controls how much randomness is injected. Because α_t is close to 1, each individual step is a small, gentle nudge — not a violent jump. Repeating this thousands of times is what produces the smooth, gradual noising sequence from Figure 1.2.

4.3 The Markov Chain Structure

Crucially, each step only depends on the step immediately before it — not on the full history. This is called a **Markov chain**:

```
q(x_1, x_2, ..., x_T | x_0)  =  q(x_1|x_0) · q(x_2|x_1) · ... · q(x_T|x_(T-1))
```

READING THIS FORMULA, TERM BY TERM

`q(x_1,...,x_T | x_0)` — the probability of the WHOLE noisy sequence happening, all at once, given we started at x_0

`= (product of small steps)` — this says the whole-sequence probability breaks apart into a simple multiplication of individual one-step probabilities

`q(x_1|x_0), q(x_2|x_1), ...` — each factor is just the single-step formula from Section 4.2, applied once per timestep

This memoryless property is what will make the mathematics in Chapter 11 tractable — exactly the same way the Markov property makes the “impossible denominator” problem from your uploaded guide solvable in small pieces instead of all at once.

Chapter 5: The Noise Schedule and the Closed-Form Shortcut

5.1 The Noise Schedule

The sequence of α_t values across all T steps is called the **noise schedule**. It is usually expressed via β_t = 1 - α_t (how much noise is added at step t).

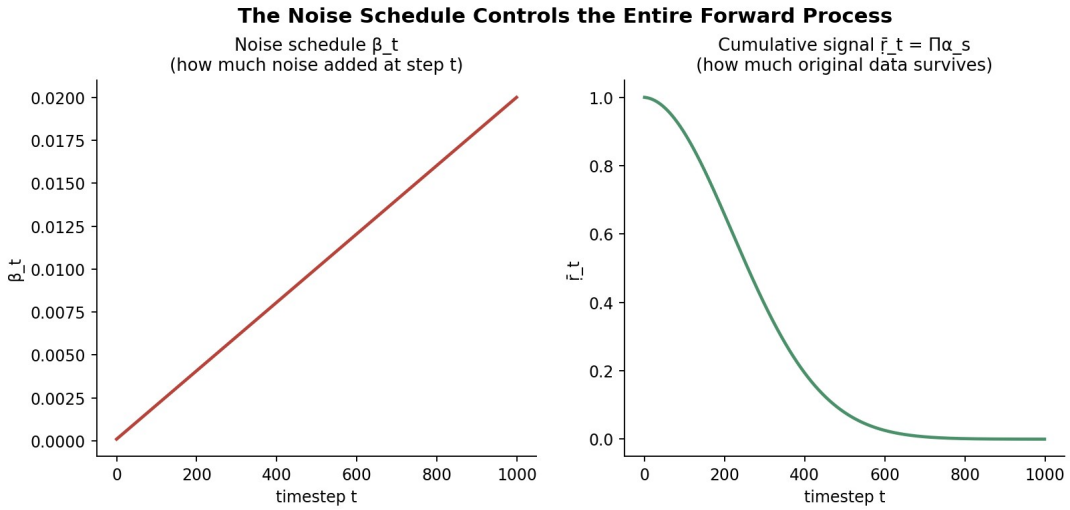


Figure 5.1 — Left: the per-step noise amount β_t. Right: ᾱ_t, the cumulative fraction of original signal remaining after t steps — it decays from 1 down toward 0.

5.2 Linear vs. Cosine Schedules

The original DDPM paper used a **linear schedule**: β_t increases evenly from β_1 = 0.0001 to β_T = 0.02. In practice this was found to destroy image information too quickly during the earliest steps, wasting a large fraction of the T steps on a signal that is already nearly pure noise. Nichol & Dhariwal's Improved DDPM (2021) proposed a **cosine schedule** instead:

```
ᾱ_t = f(t) / f(0),   where f(t) = cos( ((t/T + s) / (1+s)) · π/2 )²,    s = small offset (e.g. 0.008)
```

READING THIS FORMULA, TERM BY TERM

ᾱ_t — (“alpha-bar sub t,” pronounced “alpha bar”) how much of the original signal remains, on a 0-to-1 scale, after t steps — defined properly in Section 5.3

f(t) — a helper cosine-shaped curve, just used to build the schedule — not a diffusion quantity by itself

t / T — how far through the whole process we are, as a fraction between 0 (start) and 1 (end)

s — a small fixed offset (about 0.008) that keeps the schedule from being too aggressive right at the very start, t=0

cos(·)² — the cosine function, squared, so the curve smoothly eases from 1 down to 0 instead of dropping in a straight line

The cosine schedule keeps ᾱ_t near 1 for longer at the start and near 0 for longer at the end, spending more of the T steps in the useful "medium noise" regime where the denoising task is neither trivial nor impossible. This single change measurably improved sample quality in later models and is the default in most modern implementations.

5.3 The Trick That Makes Training Fast: Jumping Directly to Step t

If training required simulating all t steps one at a time for every training example, it would be painfully slow. Fortunately, because each step is Gaussian, and sums of Gaussians are themselves Gaussian, there is a **closed-form shortcut** allowing us to compute x_t directly from x_0 in one line, skipping every intermediate step:

```
Let ᾱ_t (alpha-bar) = α_1 · α_2 · ... · α_t     (cumulative product)

x_t = √ᾱ_t · x_0  +  √(1 - ᾱ_t) · ε,      ε ~ N(0,1)
```

READING THIS FORMULA, TERM BY TERM

ᾱ_t — multiply together every α from step 1 up through step t. Since each α is a bit less than 1, this product keeps shrinking as t grows

x_0 — the ORIGINAL clean data — notice this formula skips straight from x_0 to x_t, with no x_1, x_2, … in between

√ᾱ_t · x_0 — how much of the ORIGINAL image is still directly visible in x_t — this coefficient shrinks toward 0 as t grows

√(1-ᾱ_t) · ε — how much brand-new randomness has accumulated by step t — this coefficient grows toward 1 as t grows

ε — a SINGLE noise draw covering the combined effect of all t individual steps at once — this is exactly what makes the shortcut fast

Derivation Sketch (why this shortcut is valid)

This follows from a standard fact about Gaussians: if A ~ N(0, a^2) and B ~ N(0, b^2) are independent, then A + B ~ N(0, a^2+b^2). Applying the single-step formula twice in a row and collecting terms:

```
x_2 = √α_2·x_1 + √(1-α_2)·ε_1
    = √α_2·(√α_1·x_0 + √(1-α_1)·ε_0) + √(1-α_2)·ε_1
    = √(α_1α_2)·x_0 + [ √(α_2(1-α_1))·ε_0 + √(1-α_2)·ε_1 ]

The bracketed term is a sum of two independent Gaussians, so it collapses into a
SINGLE
Gaussian with variance = α_2(1-α_1) + (1-α_2) = 1 - α_1α_2. Repeating this
induction
all the way to step t gives exactly x_t = √ᾱ_t·x_0 + √(1-ᾱ_t)·ε.
```

READING THIS FORMULA, TERM BY TERM

`line 1 → line 2` — we simply substitute the Section 4.1 formula for x_1 into the formula for x_2 — writing x_2 in terms of x_0 instead of x_1

`line 3` — we regroup the terms: one part that scales the original x_0, and one part (in brackets) made only of noise

`the bracketed part` — a sum of two INDEPENDENT Gaussian noise terms. A well-known rule says adding two independent Gaussians gives another Gaussian, with variances simply adding together

`the final line` — after doing this same regrouping T times in a row (an induction), everything collapses to the single shortcut formula — one ε, one square-root-$\bar{\alpha}_t$ coefficient

WHY THIS MATTERS FOR TRAINING

This single formula means that during training, we never need to run the forward process step by step. We pick a random timestep t, sample one noise vector ε, and compute x_t directly in one shot. This is exactly the formula used in the training objective diagram in Chapter 11, and it is the single most important practical equation in this entire book.

5.4 Sanity Checks

- At t = 0: $\bar{\alpha}_0 = 1$, so $x_0 = 1 \cdot x_0 + 0 \cdot \varepsilon = x_0$ exactly — the original, clean data.
- As $t \to T$: $\bar{\alpha}_T \to 0$ (if the schedule is designed well), so $x_T \approx 0 \cdot x_0 + 1 \cdot \varepsilon = \varepsilon$ — pure standard Gaussian noise, with no trace of the original data left.

Chapter 6: Why Gaussian Noise, Specifically?

A natural question: why must the noise be Gaussian, and not, say, uniform noise or some other distribution?

- **Closed-form composability** — as shown in Chapter 5, sums of Gaussians remain Gaussian. This is what gives us the single-step shortcut formula. No other common noise distribution offers this convenient property.
- **A known, simple endpoint** — no matter how complex the original data distribution is, after enough forward steps, the result converges to a plain N(0,1) — a distribution we can trivially sample from to start the generation process (Chapter 13).
- **Matches the Central Limit Theorem intuition** — repeatedly adding many small independent random perturbations naturally tends toward a Gaussian shape, mirroring real physical diffusion (Brownian motion) closely.
- **Tractable reverse step** — for a sufficiently small step size, it can be shown that the true reverse of a Gaussian forward step is *also* approximately Gaussian — which is precisely the assumption that lets our neural network's output (Chapter 9) be a simple, well-behaved Gaussian too. This result traces back to Feller's theorem on the reversibility of diffusion processes.

Chapter 7: A Complete Worked Numerical Example

Equations are easiest to trust once you have pushed real numbers through them. This chapter takes one tiny, artificial example and computes every quantity from Chapters 4–6 by hand.

7.1 Setup

Suppose our “image” is a single pixel value x_0 = 0.80 (already scaled to roughly [-1,1] as diffusion models typically expect). We'll use a tiny 3-step schedule with β_1=0.10, β_2=0.20, β_3=0.30 to keep the arithmetic short.

```
α_1 = 1 - 0.10 = 0.90
α_2 = 1 - 0.20 = 0.80
α_3 = 1 - 0.30 = 0.70
```

7.2 Computing the Cumulative Products ᾱ_t

```
ᾱ_1 = 0.90                    = 0.900
ᾱ_2 = 0.90 × 0.80              = 0.720
ᾱ_3 = 0.90 × 0.80 × 0.70        = 0.504
```

7.3 Jumping Directly to x_3 Using the Shortcut

Say we draw ε = 1.20 (a somewhat-above-average noise draw). Using the closed-form shortcut from Chapter 5:

```
x_3 = √ᾱ_3 · x_0  +  √(1-ᾱ_3) · ε
    = √0.504 × 0.80  +  √0.496 × 1.20
    = 0.7099 × 0.80  +  0.7043 × 1.20
    = 0.5679  +  0.8451
    = 1.4131
```

READING THIS FORMULA, TERM BY TERM

- **√0.504 = 0.7099** — this is the SHRINK factor applied to our original pixel x_0 = 0.80
- **√0.496 = 0.7043** — this is the NOISE STRENGTH — how much of ε = 1.20 gets mixed in
- **0.5679** — the surviving contribution from the ORIGINAL signal (0.7099 × 0.80)
- **0.8451** — the contribution from pure NOISE (0.7043 × 1.20) — already larger than the signal term!

Notice x_3 = 1.4131 is quite far from the original x_0 = 0.80 — after just 3 steps with this (deliberately aggressive) toy schedule, the noise term already dominates. This is exactly the behavior Figure 1.2 shows visually.

7.4 Confirming Step-by-Step Gives the Same Distributional Result

As a check on Section 5.3's derivation, the variance of the noise coefficient in our shortcut should equal 1 - $\bar{\alpha}_3$ = 0.496. Multiplying out the step-by-step variance contributions ($\alpha_2\alpha_3(1-\alpha_1) + \alpha_3(1-\alpha_2) + (1-\alpha_3)$) gives (0.80×0.70×0.10) + (0.70×0.20) + 0.30 = 0.056 + 0.140 + 0.300 = 0.496 — matching exactly, confirming the induction from Section 5.3 is consistent.

> **TRY IT YOURSELF**
>
> Recompute x_3 with ε = 0 (the average case, no noise drawn) and confirm you get $x_3 = \sqrt{\bar{\alpha}_3} \times x_0 = 0.568$ — a pure shrinkage of the original signal toward zero, with the noise contribution removed. This is a good way to build intuition for what each term in the shortcut formula is individually responsible for.

PART III — The Reverse Process (Creation): The Actual Learning Problem

This is the technical core of the book. We derive, in full, the exact quantity a diffusion model learns, why it can be computed at all despite the intractability discussed in your uploaded guide, and how the U-Net that does the learning is actually built and trained in production systems. As in Part II, every formula is followed immediately by a plain-English breakdown of each symbol.

Chapter 8: Why We Need a Neural Network At All

8.1 The True Reverse Step is Intractable

We know the forward step q(x_t | x_(t-1)) exactly — it's just a formula. What we actually need for generation is the **reverse**: q(x_(t-1) | x_t), i.e. given a noisy image, what did the slightly-less-noisy version probably look like? By Bayes' rule (exactly as in Chapter 7 of the VAE book):

```
q(x_(t-1) | x_t)  =  q(x_t | x_(t-1)) · q(x_(t-1)) / q(x_t)
```

READING THIS FORMULA, TERM BY TERM

- **`q(x_(t-1) | x_t)`** — what we ACTUALLY want: given a noisy image x_t, what's the distribution of the slightly-cleaner version that came before it?
- **`q(x_t | x_(t-1))`** — the EASY direction — the forward formula from Chapter 4, which we already know exactly
- **`q(x_(t-1))`** — the overall ("marginal") probability of seeing THIS PARTICULAR slightly-noisy image, across the entire dataset — the troublesome term
- **`q(x_t)`** — same problem, one step further — the overall probability of this particular noisier image occurring at all

The troublesome term is q(x_(t-1)) — the marginal distribution of slightly-noisy data, which depends on the **entire, complicated real data distribution**. This is exactly the impossible Partition Function problem from your uploaded guide, showing up again here in a new disguise. We cannot compute it directly.

8.2 A Crucial Subtlety: Conditioning on x_0 Fixes This

Here is the key mathematical trick that makes diffusion models work at all. While q(x_(t-1)|x_t) alone is intractable, if we ALSO condition on the original clean data point x_0 (which we know, during training, because it's our training example), the posterior becomes exactly computable in closed form using Bayes' rule and the Gaussian forward formulas from Chapters 4–5:

```
q(x_(t-1) | x_t, x_0)  =  N( x_(t-1) ;  μ̃t(x_t, x_0),   β̃t · I )

where:
μ̃t(x_t, x_0) = ( √γ̄_(t-1)·β_t / (1-γ̄_t) )·x_0  +  ( √α_t·(1-γ̄_(t-1)) / (1-γ̄_t) )·x_t
β̃t         = ( (1-γ̄_(t-1)) / (1-γ̄_t) )·β_t
```

READING THIS FORMULA, TERM BY TERM

`q(x_(t-1)|x_t,x_0)` — the TRUE reverse step, now made computable because we've also told it the clean answer x_0 — something only available during training

`μ̃t (mu-tilde)` — the exact MEAN of this Gaussian — the best-guess value of x_(t-1) — read below as a recipe

`β̃t (beta-tilde)` — the exact VARIANCE — a single fixed number at each timestep, no learning needed for this part

`√γ̄_(t-1)·β_t / (1-γ̄_t)` — the WEIGHT given to the clean data x_0 inside the mean — how much to trust the known answer

`√α_t·(1-γ̄_(t-1)) / (1-γ̄_t)` — the WEIGHT given to the current noisy image x_t inside the mean — how much to trust where we currently are

`μ̃t(x_t,x_0) = (weight×x_0) + (weight×x_t)` — put together: the true best guess for x_(t-1) is simply a WEIGHTED BLEND of the clean data and the current noisy point — nothing more exotic than that

This is a genuinely important formula — it is the exact target our neural network is trained to approximate at every timestep. It says: the true "slightly less noisy" version is a specific, computable weighted blend of the noisy input x_t and the (training-time-only-known) clean data x_0, with a known, fixed variance β̃_t.

8.3 The Fix: Approximate It With a Neural Network

At generation time we do NOT have x_0 — that's the whole point, we're trying to create it. So we train a neural network p_θ(x_(t-1) | x_t) to output an estimate of μ̃_t **without** access to x_0, using only x_t and t. Because the true posterior derived above is exactly Gaussian, it is theoretically justified (not just a convenient assumption) for our network's output distribution to also be Gaussian — we only need it to predict a mean (and optionally a variance).

KEY IDEA

Training a diffusion model means training a single neural network, shared across all timesteps, that can look at a noisy image x_t and a timestep t, and predict something useful about how to make it slightly less noisy. What exactly it should predict is the subject of the next chapter.

Chapter 9: What Should the Network Actually Predict?

There are three mathematically equivalent choices for what the network's output represents. All three are algebraically interchangeable, but they differ in training stability and ease of implementation.

9.1 Option A: Predict the Noise ε

The most common and empirically most stable choice (used in the original DDPM paper): given a noisy x_t, predict exactly which noise vector ε was added to produce it from x_0. Since x_t = √ᾱ_t·x_0 + √(1-ᾱ_t)·ε, knowing ε lets us algebraically recover an estimate of x_0 directly:

```
x̂_0 = ( x_t - √(1-ᾱ_t)·ε_θ(x_t,t) ) / √ᾱ_t
```

READING THIS FORMULA, TERM BY TERM

`x̂_0` — the little hat means “estimate of” — this is our BEST GUESS at the original clean image, not the true x_0

`ε_θ(x_t,t)` — the neural network's predicted noise — θ (theta) denotes “the network's learned weights”, the standard symbol for a model's parameters

`x_t - √(1-ᾱ_t)·ε_θ` — take the noisy image and SUBTRACT OUT our best guess of how much noise is in it

`÷ √ᾱ_t` — un-shrink the result back to the original scale — this is just algebra, rearranging Chapter 5's shortcut formula to solve for x_0 instead of x_t

Substituting this x̂_0 into the μ̃_t formula from Chapter 8 gives the network's predicted mean directly in terms of the predicted noise — this substitution is exactly what collapses the complicated ELBO into the simple MSE loss you'll see in Chapter 11.

9.2 Option B: Predict the Clean Data x_0

Alternatively, the network can directly predict its best guess of the original clean image x_0 given the noisy x_t. This is more intuitive but tends to be less stable early in training, when x_t is almost pure noise and guessing x_0 directly is a much harder, higher-variance target.

9.3 Option C: Predict the Score ∇log p(x_t)

The score-based view (Chapter 17) frames the target as the gradient of the log-probability at x_t — literally, the arrow direction from Figure 2.1. It turns out this is mathematically just a rescaled version of the noise prediction in Option A:

```
∇log p(x_t) ≈ -ε_θ(x_t,t) / √(1-ᾱ_t)
```

READING THIS FORMULA, TERM BY TERM

`∇log p(x_t)` — ∇ ("nabla") means "gradient of." This whole term is "which direction in image-space increases the probability of this looking like real data, fastest" — the arrow from Figure 2.1

`-ε_θ(x_t,t)` — the NEGATIVE of the predicted noise — pointing away from the noise that was added, i.e. back toward clean data

`÷ √(1-ᾱ_t)` — just a rescaling factor to convert units — noise-prediction and score-prediction carry the same information, scaled differently

9.4 Option D: Predict the Velocity v (v-prediction)

A fourth parameterization, introduced by Salimans & Ho (2022), blends ε and x_0 prediction into a single quantity:

```
v = √ᾱ_t·ε - √(1-ᾱ_t)·x_0
```

READING THIS FORMULA, TERM BY TERM

`v` — the "velocity" target — not a physical velocity, just the name given to this particular blend

`√ᾱ_t·ε` — the noise contribution, weighted by how much signal remains at this timestep

`√(1-ᾱ_t)·x_0` — the clean-data contribution, weighted by how much noise has accumulated at this timestep

This turns out to be substantially more numerically stable specifically when using very few sampling steps (as in fast samplers and distillation, Chapter 14), because it avoids a division-by-near-zero problem that ε-prediction suffers from near the very end of sampling.

Prediction target	Used by	Notes
Noise ε	DDPM (most common)	Most stable in practice; simplest loss
Clean data x_0	Some DDPM variants	More intuitive, less stable early on
Score ∇log p(x_t)	Score-based / SDE models	Same information as ε, rescaled
Velocity v	Flow Matching / v-prediction / distillation	Blends ε and x_0; very stable at few steps

This book uses the noise-prediction (ε) formulation throughout, since it is the most widely implemented and the clearest starting point — exactly as DDPM (Ho et al., 2020) originally proposed.

Chapter 10: The U-Net — The Network That Does the Predicting

10.1 Why a U-Net?

The prediction network needs to take in a full-resolution noisy image and output a full-resolution noise estimate of the exact same size — an image-to-image task. The **U-Net** architecture, borrowed from image segmentation, is the standard choice: it downsamples the image through an encoder path to capture global structure, then upsamples back through a decoder path to recover fine spatial detail, with **skip connections** directly linking matching encoder/decoder resolutions so fine detail isn't lost.

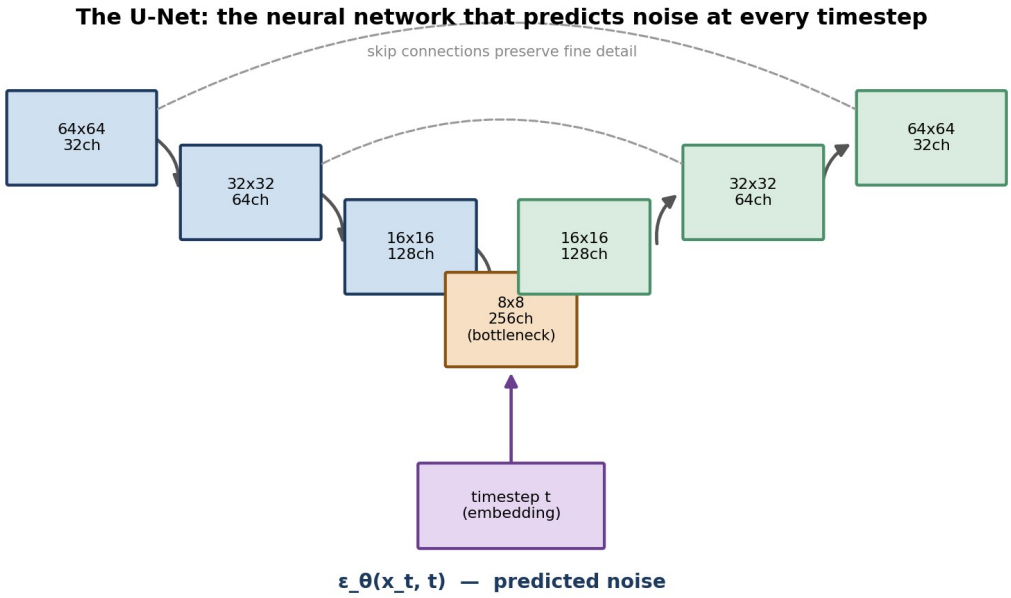


Figure 10.1 — The U-Net denoiser: a symmetric encoder-decoder with skip connections, conditioned on the timestep t.

10.2 What's Actually Inside Each Block

Each resolution level in a modern diffusion U-Net is not a single convolution, but a small stack: typically two or three **ResNet blocks** (convolution, GroupNorm, SiLU activation, with a residual/skip connection inside the block itself), followed by a **self-attention block** at the lower (coarser) resolutions only — attention is expensive, so it's reserved for the resolutions small enough to afford it, typically 32x32, 16x16 and 8x8 in a 64x64-latent Stable-Diffusion-style U-Net.

10.3 Timestep Conditioning

Since the same network must handle every noise level from nearly-clean to pure noise, it needs to know **which timestep t it is currently operating at** — removing noise from a barely-noisy image requires very different behavior than removing noise from near-total static. The timestep t is converted into a vector embedding (commonly using sinusoidal position embeddings, exactly like the positional encodings in a Transformer you already know) and injected into every ResNet block, typically by adding it to the block's intermediate feature map after a small linear projection.

10.4 Attention Layers and Cross-Attention for Conditioning

Self-attention lets the network reason about long-range relationships across the whole image (e.g. keeping both eyes symmetric and consistent), not just local neighborhoods that convolutions alone can see. For conditional models (Chapter 15's text-to-image case), a second kind of attention — **cross-attention** — is added at the same resolutions, where image features act as the query and the text-prompt's token embeddings act as the key/value pairs, exactly mirroring the encoder-decoder attention in the original Transformer architecture you already studied.

10.5 A Sense of Real-World Scale

The U-Net inside Stable Diffusion 1.5 has roughly 860 million parameters. SDXL's U-Net is substantially larger, around 2.6 billion parameters, with more attention layers and wider channels at each resolution. These are large networks by any standard, and they are run **repeatedly** — once per sampling step — which is precisely why the sampling speed discussion in Chapter 14 matters so much in production.

Chapter 11: Deriving the Training Objective, in Full

11.1 Starting From the Same ELBO as Chapter 9 of the VAE Book

A diffusion model is a hierarchical latent variable model with latents x_1, ..., x_T (Section 2.1). Applying the exact same ELBO derivation you already worked through for VAEs — just with a chain of T latents instead of one — the (negative) ELBO decomposes into a sum of KL divergence terms, one per timestep:

```
L = E_q [  D_KL(q(x_T|x_0) || p(x_T))
         + Σ_(t=2)^T D_KL(q(x_(t-1)|x_t,x_0) || p_θ(x_(t-1)|x_t))
         - log p_θ(x_0|x_1)  ]
```

READING THIS FORMULA, TERM BY TERM

`L` — the total loss we're trying to minimize — lower is better, exactly as in the VAE book

`E_q [ · ]` — "average over," taken across random training examples and random noising outcomes drawn from q

`D_KL(q(x_T|x_0) || p(x_T))` — how far the fully-noised training data is from a perfect standard Gaussian — this term has NO learnable parameters and is dropped during training

`Σ_(t=2)^T` — "sum this up, once for every timestep from 2 to T" — the sigma is just repeated addition

`D_KL(q(x_(t-1)|x_t,x_0) || p_θ(x_(t-1)|x_t))` — for each timestep: how far is our network's guess (right side) from the TRUE answer derived in Chapter 8 (left side)? This is the real workhorse term

`- log p_θ(x_0|x_1)` — a separate term just for the very last step, turning the final noisy version back into the actual output image

The first term is constant (no learnable parameters — both sides are fixed once the schedule is chosen) and can be dropped during training. The last term is a separate reconstruction term for the final step. The middle summation is the real workhorse: each D_KL term compares our network's prediction to the exact, closed-form posterior q(x_(t-1)|x_t,x_0) derived in Section 8.2.

11.2 KL Divergence Between Two Gaussians With Matched Variance

Since both q(x_(t-1)|x_t,x_0) and p_θ(x_(t-1)|x_t) are Gaussians, and DDPM fixes the network's output variance to a known constant (matching $\tilde{\beta}_t$ from Section 8.2, rather than learning it), the KL divergence between two Gaussians with equal variance collapses to a simple squared distance between their means:

```
D_KL( N(μ̃t, β̃t) || N(μ_θ, β̃t) )  =  ( 1 / (2·β̃t) ) · || μ̃t - μ_θ ||²
```

READING THIS FORMULA, TERM BY TERM

N(μ̃_t, β̃_t) — the TRUE Gaussian from Chapter 8 — correct mean, correct variance

N(μ_θ, β̃_t) — our NETWORK's Gaussian guess — same (fixed) variance, but its own predicted mean μ_θ

1 / (2·β̃_t) — just a constant weighting factor — doesn't depend on the network's weights, so it mostly just rescales the loss

|| μ̃_t - μ_θ ||² — ordinary squared distance between the true mean and the predicted mean — this is the part that actually drives learning

11.3 The Remarkable Simplification

Substituting the noise-prediction form of both μ̃_t and μ_θ (Section 9.1) into this expression, the shared coefficients cancel almost entirely, and Ho et al. (2020) showed the whole per-timestep loss term reduces to a weighted squared error between the true and predicted noise. Dropping the remaining (empirically unhelpful) weighting term gives the final, famously simple training loss:

$$L_{simple}(\theta) = E_{(x_0, \varepsilon, t)} \left[\| \varepsilon - \varepsilon_\theta(x_t, t) \|^2 \right]$$

$$\text{where} \quad x_t = \sqrt{\bar{r}_t} \cdot x_0 + \sqrt{(1-\bar{r}_t)} \cdot \varepsilon$$

READING THIS FORMULA, TERM BY TERM

L_simple(θ) — the actual loss implemented in code — “simple” because of how much complexity from Sections 11.1–11.2 collapsed away

E_(x0, ε, t) [·] — average this over: random training images x_0, random noise draws ε, and random timesteps t — exactly the three random ingredients in the training loop of Chapter 12

ε — the TRUE noise we actually added — known, because we sampled it ourselves during training

ε_θ(x_t,t) — the network's PREDICTED noise — unknown until the network gives its guess

|| ε - ε_θ(x_t,t) ||² — ordinary squared error between the true and predicted noise — the exact same kind of loss as any regression problem

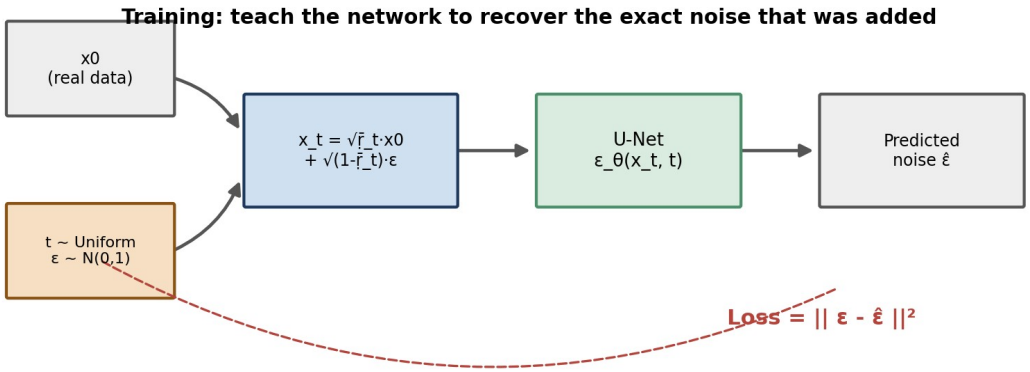


Figure 11.1 — The full training step: sample real data, a random timestep, and noise; build x_t with the Chapter 5 shortcut; ask the U-Net to recover the noise; compare to the true noise.

KEY IDEA

All of that careful ELBO derivation across a chain of hundreds of latent variables — including a genuinely nontrivial closed-form Gaussian posterior — reduces, in practice, to one of the simplest loss functions in all of deep learning: predict the noise, compare with mean squared error. This simplicity is a major reason diffusion models train so reliably compared to GANs, and it's a striking example of how a rigorous derivation can justify what looks, at first glance, like an almost too-simple training recipe.

Chapter 12: Full Training Algorithm and Production Code

12.1 The Training Algorithm, in Words

1. Take a real training image x_0.
2. Sample a random timestep t, uniformly between 1 and T.
3. Sample random Gaussian noise ε.
4. Build the noisy image x_t directly using the closed-form shortcut from Chapter 5.
5. Feed x_t and t into the U-Net; get its predicted noise $\hat{\varepsilon}$.
6. Compute the loss $|| \varepsilon - \hat{\varepsilon} ||^2$ and backpropagate, updating the U-Net's weights.
7. Repeat for millions of (image, timestep, noise) combinations until the network reliably predicts noise at every noise level.

12.2 PyTorch Implementation

```
import torch
import torch.nn as nn
import torch.nn.functional as F

T = 1000
betas = torch.linspace(1e-4, 0.02, T)
alphas = 1.0 - betas
alpha_bars = torch.cumprod(alphas, dim=0)

def forward_diffusion_sample(x0, t, noise):
    """Directly compute x_t from x_0 using the closed-form shortcut (Chapter
5)."""
    sqrt_alpha_bar = alpha_bars[t].sqrt().view(-1, 1, 1, 1)
    sqrt_one_minus_alpha_bar = (1 - alpha_bars[t]).sqrt().view(-1, 1, 1, 1)
    return sqrt_alpha_bar * x0 + sqrt_one_minus_alpha_bar * noise

unet = UNet()  # a U-Net that takes (x_t, t) and outputs predicted noise, same
shape as x_t
optimizer = torch.optim.Adam(unet.parameters(), lr=2e-4)

for x0, _ in dataloader:                                  # labels ignored, same
as an Autoencoder
    batch_size = x0.shape[0]
    t = torch.randint(0, T, (batch_size,))                # random timestep per
example
    noise = torch.randn_like(x0)                          # ε ~ N(0,1)

    x_t = forward_diffusion_sample(x0, t, noise)
    predicted_noise = unet(x_t, t)

    loss = F.mse_loss(predicted_noise, noise)             # L_simple from
Chapter 11
    optimizer.zero_grad()
    loss.backward()
```

```
    optimizer.step()
```

12.3 Two Production Techniques Worth Knowing

Real training pipelines almost always add two techniques not shown above, both simple but important:

- **Exponential Moving Average (EMA) of weights** — maintain a second copy of the U-Net's weights that is updated as a slow-moving average of the actual training weights (typically with decay 0.999 or 0.9999) after every step: `ema_weights = decay*ema_weights + (1-decay)*current_weights`. The EMA weights are used for sampling/evaluation, since they are noticeably more stable and produce better samples than the raw, noisier training weights at any single step.
- **Mixed-precision training** — running the forward/backward pass in 16-bit floating point (with a 32-bit master copy of weights for the optimizer update) roughly halves memory usage and often speeds up training significantly on modern GPUs, with negligible quality loss when implemented carefully (e.g. via PyTorch's `torch.cuda.amp`).

Notice how closely the core loop mirrors the VAE training loop from Chapter 11 of the previous book: sample something random, run it through a network, compare the output to a known target with MSE, backpropagate. The conceptual machinery is identical — only the specific quantities being compared, and these two extra production-hardening techniques, have changed.

PART IV — Sampling: Turning Noise Into New Data

Training gives us a network that can predict noise. This part covers how that single capability is turned into a full generation procedure, how to make that procedure dramatically faster, and how to steer it toward what you actually want.

Chapter 13: The Sampling Loop

13.1 Generation, Step by Step

Once the U-Net is trained, generating a brand-new sample is simply running the reverse process (Figure 1.3) from scratch, starting from pure noise:

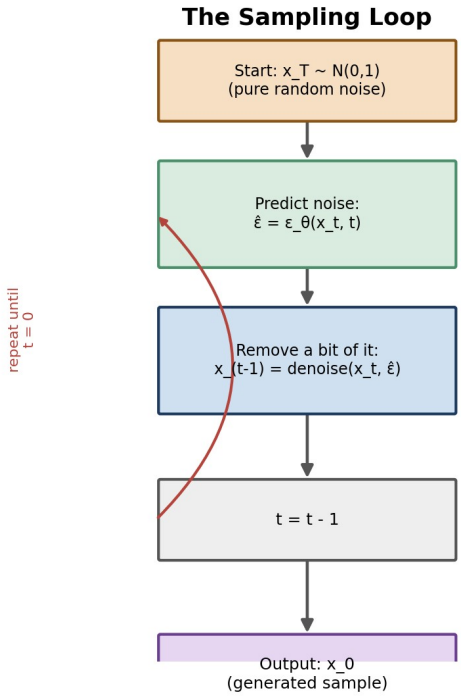


Figure 13.1 — The sampling loop: start from pure noise, repeatedly predict and remove a little noise, until a clean sample emerges.

1. Sample x_T directly from the known prior N(0,1) — pure random noise, the same shape as a real image.
2. For t = T down to 1: feed x_t and t into the trained U-Net to get predicted noise $\hat{\varepsilon}$.
3. Use $\hat{\varepsilon}$ to compute the mean μ_θ (using the formula from Section 9.1, substituted into Section 8.2's $\tilde{\mu}_t$ formula), then sample $x_{t-1} \sim N(\mu_\theta, \tilde{\beta}_t)$ — adding a small amount of fresh noise, except at the very final step.
4. After looping all the way down to t = 0, the final x_0 is a brand-new, generated sample.

KEY IDEA

Sampling from a diffusion model is inherently iterative and sequential — unlike a VAE or GAN, which generate an output in a single forward pass. This is the direct cause of diffusion models' main weakness (slow generation) and is the entire motivation for Chapter 14.

13.2 PyTorch Sampling Code

```
@torch.no_grad()
def sample(unet, shape, T, betas, alphas, alpha_bars):
    x_t = torch.randn(shape)                    # start from pure noise, x_T
    for t in reversed(range(T)):
        predicted_noise = unet(x_t, torch.tensor([t]))
        alpha_t = alphas[t]
        alpha_bar_t = alpha_bars[t]
        beta_t = betas[t]

        # Estimate the mean of x_(t-1) from x_t and the predicted noise
        coef = (1 - alpha_t) / (1 - alpha_bar_t).sqrt()
        mean = (1 / alpha_t.sqrt()) * (x_t - coef * predicted_noise)

        if t > 0:
            noise = torch.randn_like(x_t)
            x_t = mean + beta_t.sqrt() * noise    # add a little fresh noise back
in
        else:
            x_t = mean                            # final step: no more noise
added
    return x_t                                    # this is x_0, the generated
sample
```

Chapter 14: Faster Sampling — DDIM and Beyond

14.1 The Speed Problem

The original DDPM sampling procedure requires T sequential U-Net passes — typically T = 1000. Each pass is a full neural network forward pass through a large U-Net (recall Section 10.5: up to billions of parameters), so generating a single image can take many seconds or longer. This is the primary practical downside of diffusion models compared to a VAE or GAN's single-pass generation.

14.2 DDIM: A Non-Markovian Reformulation

DDIM (Song, Meng & Ermon, 2021) makes a key observation: the *training* objective (Chapter 11) only depends on the marginal distributions q(x_t|x_0) at each timestep, not on the specific Markov chain structure used to define them. This means we are free to define a **different family** of forward processes sharing the same marginals but a different (non-Markovian) joint structure — and correspondingly, a different, more flexible reverse sampler, without retraining anything. The DDIM update rule is:

```
x_(t-1) = √ᾱ_(t-1) · x̂_0  +  √(1-ᾱ_(t-1) - σ_t²) · ε_θ(x_t,t)  +  σ_t · z,
z~N(0,1)
```

READING THIS FORMULA, TERM BY TERM

`x̂_0` — our current best guess at the clean image, computed exactly as in Section 9.1

`√ᾱ_(t-1) · x̂_0` — re-noise that guess back up to the (lower) noise level of step t-1, using the SAME closed-form shortcut formula from Chapter 5, just run partway

`√(1-ᾱ_(t-1) - σ_t²) · ε_θ` — add back in a controlled amount of the SAME predicted-noise direction — this is what keeps the trajectory smooth and consistent step to step

`σ_t · z` — an optional extra dash of FRESH randomness — z is a new random draw, and σ_t controls how much of it to add

`σ_t = 0 (special case)` — setting this free parameter to zero removes ALL extra randomness, making the entire sampling path fully deterministic — this is "DDIM proper," discussed next

Setting σ_t = 0 for every step makes the entire sampling process **fully deterministic**: the same starting noise x_T always produces exactly the same output image, with no randomness injected along the way. This deterministic setting is DDIM proper, and it is what allows skipping large groups of timesteps at once (e.g. jumping straight from t=1000 to t=950) without the accumulated-error problems that skipping steps in the original stochastic DDPM sampler would cause.

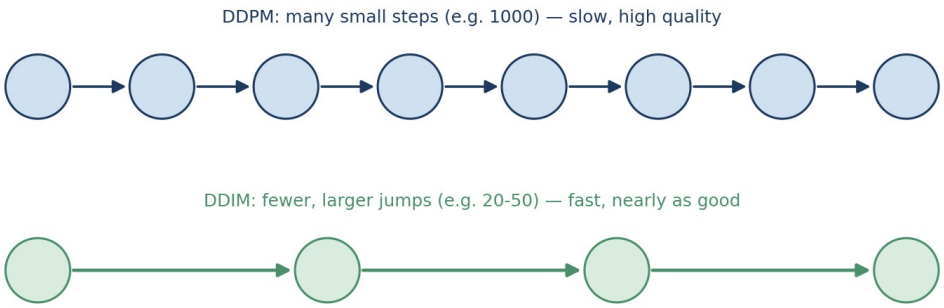


Figure 14.1 — DDPM takes many small, cautious steps; DDIM takes far fewer, larger steps toward the same destination.

14.3 A Useful Side Effect: Meaningful Latent Codes

Because DDIM sampling is deterministic, the starting noise x_T functions like a genuine latent code for the generated image — directly analogous to the latent z in a VAE. This enables latent-space interpolation between two generated images (exactly like Chapter 18 of the VAE book), and is the mechanism behind many image-editing tools built on diffusion models: invert a real image to find its x_T, edit the prompt, then re-run DDIM sampling from that same x_T.

14.4 Further Speedups

- **DPM-Solver / higher-order ODE solvers** — treat sampling as solving a differential equation (Chapter 17) and apply numerical methods (like Runge-Kutta) that need far fewer function evaluations for the same accuracy, often reaching good quality in 10–20 steps.
- **Distillation** — train a second, smaller/faster model to mimic what many steps of the original model would produce in a single step (progressive distillation halves the required steps repeatedly).
- **Consistency Models** — train a model that can jump directly from any noise level straight to a clean sample in one or very few steps (Chapter 19).

Chapter 15: Controlling What Gets Generated

15.1 Classifier Guidance

The earliest approach to controllable generation (Dhariwal & Nichol, 2021) trains a separate classifier p(y|x_t) that can recognize a desired attribute (e.g. “cat”) even in noisy images, and uses its gradient ∇log p(y|x_t) to nudge the sampling process toward that class at every step, added directly to the predicted score. This works but requires training and maintaining an entirely separate, noise-aware classifier network alongside the main U-Net.

15.2 Classifier-Free Guidance (CFG)

The modern standard approach avoids a separate classifier entirely. During training, the conditioning information y (e.g. a text prompt) is randomly dropped out some percentage of the time (typically 10-20%), so the same U-Net learns to predict noise **both** conditionally, ε_θ(x_t,t,y), and unconditionally, ε_θ(x_t,t,∅). At sampling time, we combine both predictions, exaggerating the difference to push generation more strongly toward the prompt:

```
ε_final = ε_uncond + w · ( ε_cond - ε_uncond )
```

READING THIS FORMULA, TERM BY TERM

- **`ε_uncond`** — the network's noise prediction with NO prompt given (y = ∅, the empty/null condition)
- **`ε_cond`** — the network's noise prediction WITH the prompt given
- **`ε_cond - ε_uncond`** — the DIFFERENCE that the prompt alone made — the “direction” the prompt is pushing generation in
- **`w`** — the guidance weight — a dial you choose at sampling time (typically 1 to 15). Bigger w = push harder in that prompt direction
- **`w · ( · )`** — exaggerate that prompt-direction difference by a factor of w before adding it back
- **`ε_final`** — the actual noise prediction used for this sampling step, after the guidance boost has been applied

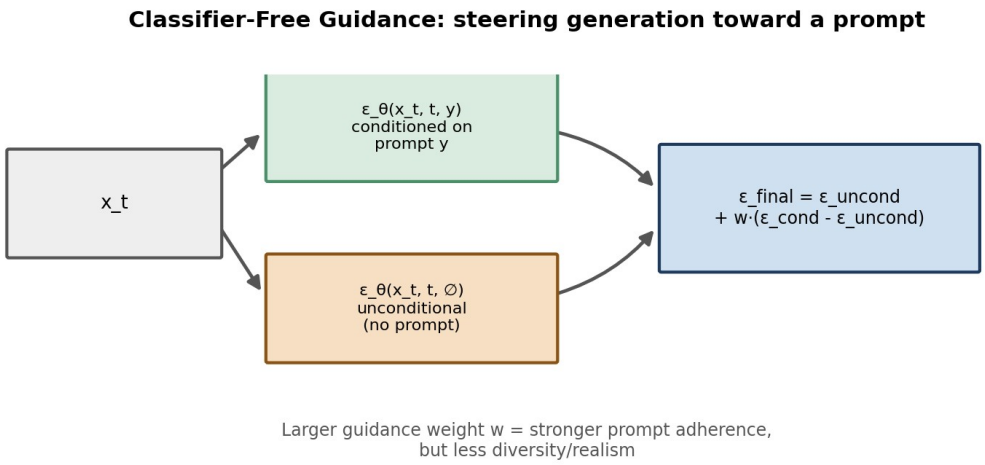


Figure 15.1 — Classifier-free guidance amplifies the difference between conditional and unconditional predictions, steering generation toward the prompt.

The guidance weight w is a dial: w = 1 means pure conditional generation with no boost, while larger values (e.g. w = 7–15, typical for text-to-image models) push much harder toward matching the prompt, usually at some cost to output diversity and occasionally to realism if pushed too far (over-saturated colors, exaggerated contrast are the classic symptoms of w set too high).

15.3 Negative Prompts

A practical extension: instead of always using the purely unconditional ε_uncond as the subtracted term, substitute in ε_θ(x_t,t,y_negative) — the prediction conditioned on a "negative prompt" describing what to avoid (e.g. "blurry, low quality"). The same CFG formula then pushes generation simultaneously toward the positive prompt and away from the negative one, which is exactly what the "negative prompt" box in tools like Stable Diffusion WebUIs is doing under the hood.

15.4 Guidance Rescaling

A known side effect of large guidance weights is that the predicted noise's magnitude grows unnaturally large, pushing pixel values out of their expected range and causing over-saturation. Guidance rescaling (Lin et al., 2023) renormalizes ε_final after the CFG combination step to match the statistics of ε_cond, substantially reducing this artifact at high guidance weights — a small but practically important fix widely adopted in modern pipelines.

PART V — The Modern Landscape

This part covers how the core DDPM idea from Parts II–IV has evolved into the systems actually deployed today, the deeper mathematical unification behind them, and how to rigorously evaluate a diffusion model once you've trained one.

Chapter 16: Latent Diffusion Models (Stable Diffusion)

16.1 The Problem With Running Diffusion on Raw Pixels

Running the full forward/reverse process directly on, say, a 1024×1024 pixel image is extremely expensive — every one of the (possibly hundreds of) U-Net passes must process the entire high-resolution image. Recall from Chapter 14 of the VAE book: an ordinary VAE is very good at compressing an image into a much smaller latent representation while preserving the important content.

16.2 The Latent Diffusion Idea

Latent Diffusion (Rombach et al., 2022 — the technique behind Stable Diffusion) combines both books you now know directly: first train a VAE encoder/decoder to compress images into a small latent grid (typically downsampling by a factor of f=8 per side, e.g. 512×512×3 → 64×64×4, roughly a 48x reduction in total size). Then, run the **entire forward and reverse diffusion process inside that small latent space** instead of on raw pixels. Finally, decode the generated latent back into a full-resolution image using the VAE's decoder.

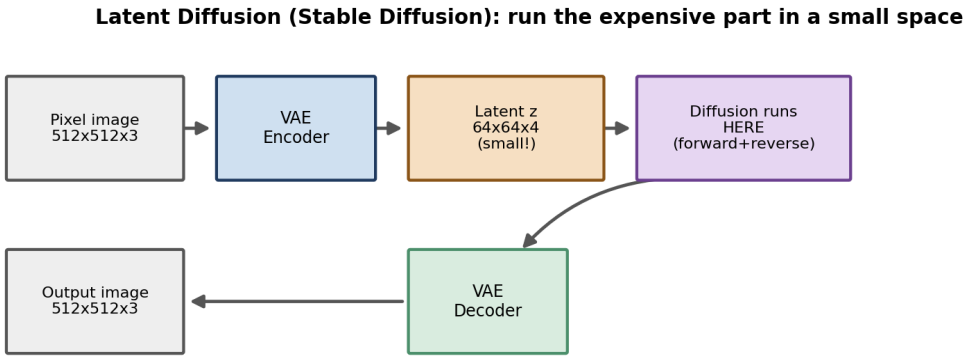


Figure 16.1 — Latent Diffusion: the expensive iterative diffusion process runs entirely inside a small, VAE-compressed latent space.

16.3 The VAE Used Here Is Slightly Different

The VAE used in Latent Diffusion is trained with a few additions beyond the plain reconstruction + KL loss from the VAE book: a **perceptual loss** (comparing deep features from a pretrained network rather than raw pixels, so reconstructions look right to a human even if pixels don't match exactly) and a lightweight **adversarial loss**

(a small discriminator, borrowing the GAN idea, pushing reconstructed patches to look locally realistic). The KL term's weight is kept very small — just enough to keep the latent space reasonably well-behaved for the diffusion model that will run inside it, without forcing heavy compression-driven blurriness.

16.4 How the Major Public Models Differ

Model	Latent resolution	Notable differences
Stable Diffusion 1.5	64×64×4 (f=8)	860M-parameter U-Net; CLIP text encoder
SDXL	128×128×4 (f=8, larger base res)	2.6B-parameter U-Net; dual text encoders; refiner stage
Stable Diffusion 3	Latent + Transformer backbone	Replaces U-Net with a Diffusion Transformer (DiT); uses Flow Matching (Ch. 18)

KEY IDEA

Latent Diffusion is not a new theory — it is a direct, practical combination of the two books you now know: a VAE handles compression (fast, one-shot), and a Diffusion Model handles the actual generative process (iterative, high-quality), each doing the part it's best at.

Chapter 17: The Score-Based / SDE Unification

17.1 A Continuous-Time View

So far we described diffusion using a fixed number of discrete steps, T. An equivalent and more general way to describe the exact same process treats t as a **continuous** variable, and expresses the forward process as a stochastic differential equation (SDE):

```
dx = f(x,t)·dt  +  g(t)·dw
```

READING THIS FORMULA, TERM BY TERM

`dx` — an infinitesimally tiny CHANGE in x — the continuous-time version of "x_t minus x_(t-1)" from the discrete formula

`f(x,t)·dt` — the DRIFT term — a smooth, predictable pull (matching the "shrink toward zero" behavior from Chapter 4) over a tiny time increment dt

`g(t)` — controls how MUCH random noise gets injected per unit time — the continuous-time analogue of √(1-α_t) from Chapter 4

`dw` — an infinitesimal random "wiggle" (a Wiener process / Brownian motion increment) — the continuous-time analogue of ε from Chapter 4

Here f(x,t) is a deterministic drift term (pulling the signal toward zero, matching the VP schedule from Chapter 3), g(t) controls how much random noise (dw, a Wiener process / Brownian motion increment) is injected per unit time, and this single continuous equation generalizes the discrete step formula from Chapter 4 to the limit of infinitely many, infinitesimally small steps.

17.2 The Reverse-Time SDE and the Score Function

Anderson (1982) proved a remarkable general result: any forward SDE of this form has a corresponding **reverse-time SDE** that exactly undoes it, and that reverse SDE requires exactly one additional piece of information: the score function ∇log p_t(x), i.e. the gradient of the log-probability at every point and noise level — precisely the vector field from Figure 2.1.

```
dx = [ f(x,t) - g(t)²·∇log p_t(x) ] dt  +  g(t)·dŵ      (running t backward)
```

READING THIS FORMULA, TERM BY TERM

`f(x,t)` — the SAME drift term as the forward SDE

`- g(t)²·∇log p_t(x)` — a CORRECTION term — subtract off an amount proportional to the score, which is exactly what steers the process back toward real data instead of away from it

`dŵ` — a Brownian motion increment running in REVERSE time (the tilde marks it as belonging to the reverse

process)

`∇log p_t(x)` — the score function AT TIME t — this is the one unknown piece that our trained network is standing in for

17.3 The Probability Flow ODE

A further remarkable result: there exists an ordinary differential equation (no randomness at all) whose solutions trace the exact same marginal distributions p_t(x) as the stochastic reverse SDE above, at every t:

```
dx = [ f(x,t) - ½·g(t)²·∇log p_t(x) ] dt      (deterministic — the Probability Flow ODE)
```

READING THIS FORMULA, TERM BY TERM

`(no dw or dŵ term)` — notice there is NO randomness left in this equation at all — that's the whole point, it's an ODE (ordinary), not an SDE (stochastic)

`½·g(t)²·∇log p_t(x)` — the same score-based correction as the reverse SDE, but at HALF strength — the exact factor needed so this deterministic path visits the same distribution of points, without needing fresh randomness at every step

This is precisely the continuous-time generalization of DDIM's deterministic sampling from Section 14.2, and it is what allows advanced numerical ODE solvers (DPM-Solver and similar, Section 14.4) to be applied directly, since solving ODEs accurately with few steps is a mature, well-studied numerical analysis problem.

17.4 Score Matching = Noise Prediction

Training a network to predict the score is mathematically equivalent (up to the known rescaling factor from Section 9.3) to training a network to predict noise, exactly as in Chapter 11. This is not a coincidence — it is a precise mathematical equivalence.

17.5 One Underlying Idea, Three Historical Names

Recent theoretical work (including the “The Principles of Diffusion Models” monograph, from the sources you linked) formally unifies three approaches that were historically developed somewhat independently: the **variational view** (DDPM, which we derived in Chapter 11), the **score-based view** (this chapter), and the **flow-based view** (Chapter 18). All three are shown to be learning the same underlying object — a time-dependent vector field that reverses a fixed corruption process — just parameterized and derived differently, with the Probability Flow ODE and the Fokker-Planck equation (Section 1.1) as the connective tissue guaranteeing they are all mathematically consistent with each other.

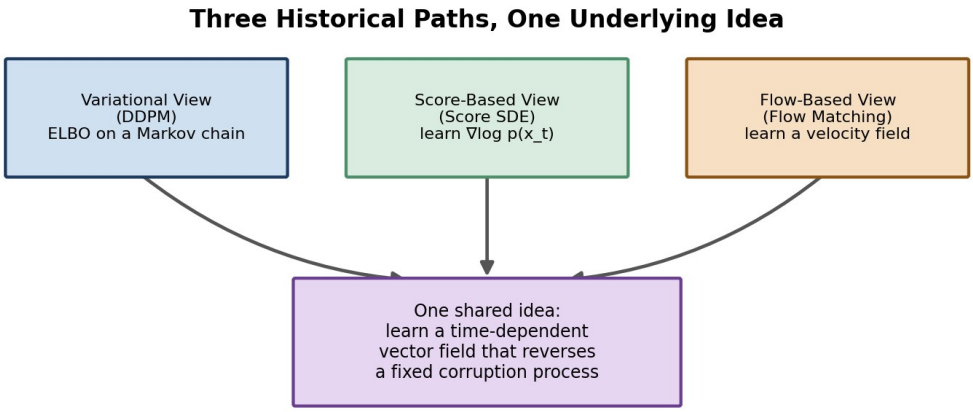


Figure 17.1 — Three historically separate research threads converge on the same core mathematical object.

Chapter 18: Flow Matching

18.1 A More General Starting Point

Flow Matching (Lipman et al., 2023) is a more recent, closely related formulation. Instead of framing generation as denoising a specific Gaussian diffusion process, it frames it directly as learning a smooth, continuous path (a "flow") from a simple noise distribution to the data distribution, and trains a network to predict the *velocity* of this flow at any point and time. A particularly simple and popular choice is the straight-line ("rectified" / "conditional optimal transport") path between a noise sample x_0′ ~ N(0,1) and a data sample x_1′:

```
x_τ = (1-τ)·x_0′ + τ·x_1′,        τ ∈ [0,1]
target velocity: u = x_1′ - x_0′      (constant along the whole straight-line
path)

Loss:  L_CFM(θ) = E [ || u - v_θ(x_τ, τ) ||² ]
```

READING THIS FORMULA, TERM BY TERM

- **τ** — (tau) plays the same role as t/T did before — a progress marker from 0 (pure noise) to 1 (real data)
- **x_0′** — a random NOISE starting point (the prime just distinguishes this from x_0 = clean data used elsewhere in the book)
- **x_1′** — a real DATA point — the destination of the path
- **x_τ = (1-τ)x_0′ + τx_1′** — literally just a straight line, linearly interpolating between the noise point and the data point
- **u = x_1′ - x_0′** — the TARGET velocity — how fast and in what direction we're moving along that straight line. Notice it never changes along the path, since the path is straight
- **v_θ(x_τ,τ)** — the network's PREDICTED velocity at this point and progress-marker
- **L_CFM(θ)** — Conditional Flow Matching loss — same shape as L_simple from Chapter 11, just comparing velocities instead of noise

Critically, Flow Matching does not require the forward process to be Gaussian diffusion specifically — it generalizes to a much broader family of valid paths between noise and data, and the straight-line path in particular tends to allow high-quality generation in far fewer sampling steps than classic DDPM, because the ODE trajectories are close to straight lines rather than the curved paths a standard diffusion schedule produces.

18.2 Where This Is Heading

Modern systems (including Stable Diffusion 3, per Section 16.4) increasingly use Flow Matching or hybrid variational/flow objectives instead of the original DDPM formulation, often paired with a Diffusion

Transformer (DiT) backbone instead of a convolutional U-Net, while keeping the same overall iterative sampling structure you already understand from Part IV.

Chapter 19: Consistency Models — Toward One-Step Generation

19.1 The Self-Consistency Property

A major recent research direction (Song et al., 2023) asks: can we get diffusion-quality samples without hundreds of sequential steps at all? **Consistency Models** train a function f_θ(x_t, t) with a special property called **self-consistency**: for any two points x_t and x_t′ lying on the *same* Probability Flow ODE trajectory (Section 17.3), the function must map both to the exact same output:

```
f_θ(x_t, t) = f_θ(x_t′, t′)   for all t, t′ on the same ODE trajectory,   with
f_θ(x_0, 0) = x_0
```

READING THIS FORMULA, TERM BY TERM

`f_θ(x_t, t)` — the consistency function — takes ANY noisy point and ANY timestep, and directly outputs a clean-data guess

`x_t and x_t′` — two DIFFERENT points, at two different noise levels, that happen to sit on the same underlying denoising trajectory

`f_θ(x_t,t) = f_θ(x_t′,t′)` — the key constraint: no matter WHERE along the same path you start from, the function must land on the SAME final answer

`f_θ(x_0,0) = x_0` — a boundary condition — at the clean end of the trajectory (t=0), the function must simply return the input unchanged, anchoring the whole function to reality

Once trained this way, generation can be done in a single network call: sample x_T ~ N(0,1) and directly compute f_θ(x_T, T) — no loop required at all — while retaining much of the sample diversity and quality that made diffusion models attractive in the first place. A small number of additional refinement calls (2–4) can be used for higher quality, directly resolving the speed-vs-quality trade-off discussed in Chapter 14.

19.2 Two Ways to Train One

- **Consistency Distillation** — start from an already-trained diffusion model, and train the consistency function to match what the original model's (many-step) ODE solver would eventually produce.
- **Consistency Training** — train a consistency model directly from data, with no pretrained diffusion teacher required, using an approximation of the self-consistency property enforced between adjacent noise levels.

Chapter 20: Evaluating Diffusion Models

20.1 Why Evaluation Is Hard

Exactly as with VAEs, judging a generative model means judging an entire learned distribution, not a single number. A model that memorizes training data would reconstruct perfectly while being a useless generator — no single metric tells the whole story, and production teams typically track several in parallel.

20.2 Fréchet Inception Distance (FID)

The most widely used image-quality metric, shared with GANs (recall the VAE book's evaluation chapter). Real and generated images are passed through a pretrained Inception network, features are extracted from an intermediate layer, and the distance between the two resulting feature distributions (modeled as Gaussians) is computed:

```
FID = || μ_real - μ_fake ||² + Tr( Σ_real + Σ_fake - 2·(Σ_real · Σ_fake)^(1/2) )
```

READING THIS FORMULA, TERM BY TERM

`μ_real, μ_fake` — the AVERAGE feature vector (from the pretrained Inception network) across real images, and across generated images, respectively

`|| μ_real - μ_fake ||²` — how far apart the two “centers” are — a big gap here means generated images look systematically different from real ones

`Σ_real, Σ_fake` — the COVARIANCE (spread/shape) of the real and generated feature distributions — captures diversity, not just average appearance

`Tr( · )` — the “trace” of a matrix — just add up its diagonal entries; a standard way to turn a matrix comparison into a single number

`(Σ_real·Σ_fake)^(1/2)` — a matrix square root — a technical ingredient making this a proper distance between two full Gaussian distributions, not just their means

Lower FID means the generated images' feature statistics more closely match real images' — capturing both realism and diversity in one number.

20.3 CLIP Score (for Text-Conditioned Models)

For text-to-image models specifically, CLIP Score measures how well a generated image matches its text prompt, by encoding both the image and the prompt with a pretrained CLIP model and computing the cosine similarity between their embeddings. This directly measures **prompt faithfulness**, which FID alone does not capture — a model can produce beautiful, realistic images that simply ignore the prompt, and FID would not necessarily penalize this.

20.4 Precision and Recall for Generative Models

A single FID number can hide an important trade-off: a model might produce very realistic images that only cover a narrow slice of the true data distribution (high “precision”, low “recall” / diversity), or the reverse. Precision/Recall metrics for generative models (Kynkäänniemi et al., 2019) separately estimate whether generated samples fall within the real data manifold (precision) and whether generated samples collectively cover the full extent of the real data manifold (recall) — directly analogous to classification precision/recall, but applied to distribution coverage.

20.5 Summary Checklist

Check	What it tells you
FID	Overall realism and diversity of generated samples
CLIP Score	How well generations match their text prompt (conditional models)
Precision / Recall	Realism vs. diversity trade-off, separated
Sampling steps vs. quality curve	Where a given sampler/model sits on the speed-quality trade-off (Chapter 14)
Qualitative visual inspection	Artifacts, mode collapse, or failure cases no single metric flags reliably

PART VI — Diffusion Models for Robotics and Autonomous Systems

This final part connects everything above directly to your own work in robotics simulation, ROS 2, and autonomous systems. Diffusion models are no longer just an image-generation tool — they have become a serious, widely-adopted technique for generating robot behavior itself.

Chapter 21: Diffusion Policy — Robot Control as Denoising

21.1 The Core Idea

A robot's job is often framed as: given the current observation (camera image, joint states, etc.), predict what action to take next. **Diffusion Policy** (Chi et al., 2023) reframes this problem using exactly the machinery from Parts II–IV of this book — except the "image" being generated is not a picture, it is a **sequence of future robot actions**.

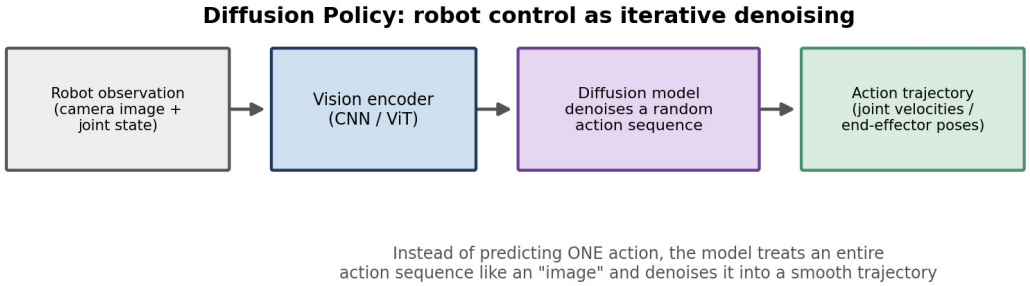


Figure 21.1 — Diffusion Policy: a robot's observation conditions a diffusion process that denoises a random action sequence into a smooth, coordinated trajectory.

21.2 Why This Works Better Than Predicting One Action Directly

Many real-world robot behaviors are **multimodal** — there are several equally valid ways to complete a task (e.g. a robot arm could go around an obstacle from the left or the right). A network trained to directly regress one "best" action (with ordinary MSE loss) tends to average these possibilities together, producing a blurry, physically invalid compromise action — exactly the same blurriness problem the VAE book discussed in the context of pixel-wise reconstruction loss. A diffusion model, by contrast, can represent complex, multimodal distributions over action sequences naturally, since it never has to commit to a single averaged answer — it samples one full, coherent, consistent solution from the learned distribution.

21.3 Receding-Horizon Control and Action Chunking

The original Diffusion Policy paper defines three key time windows that are worth knowing precisely, since they show up directly in any real implementation:

Symbol	Meaning	Typical value
T_o	Observation horizon — how many past timesteps of observation are fed in	2
T_p	Prediction horizon — how many future action timesteps the diffusion model generates at once	16
T_a	Execution horizon — how many of those predicted actions are actually executed before re-planning	8

After executing T_a actions, the robot re-observes the world and runs the full denoising process again with fresh observations — this is **receding-horizon control**, the same core idea used in Model Predictive Control (which you have direct experience with from your TurtleBot3 MPC work), just with a diffusion model in place of an explicit dynamics-model-based optimizer. Predicting more actions than are executed (T_p > T_a) gives the policy a chance to plan coherent, temporally-consistent motion, while still re-planning often enough to react to a changing environment.

21.4 Practical Architecture

Diffusion Policy component	Image-diffusion equivalent
Robot observation (image + proprioception)	Conditioning signal y (like a text prompt)
Action sequence, shape (T_p, action_dim)	The “image” being denoised
1D temporal CNN or Transformer	The 2D spatial U-Net from Chapter 10
Denoising loop at inference time	Exactly the sampling loop from Chapter 13

The observation is encoded (often with a CNN or a Vision Transformer — the same vision backbones you have used in your Cognex inspection and VLA work) and injected as a conditioning signal into the denoising network at every step, exactly like the timestep and text-prompt conditioning from Chapters 10 and 15. The original paper found a 1D temporal convolutional U-Net (a direct 1D analogue of Chapter 10's 2D image U-Net) to be a strong, efficient default, with a Transformer-based variant offering an alternative for longer horizons.

21.5 Reported Results

Across the original paper's benchmark of simulated and real-world manipulation tasks, Diffusion Policy improved average success rate by roughly 46 percent over the next-best prior method at the time (a mix of LSTM-GMM, IBC, and BET baselines), with the largest gains specifically on tasks with high multimodality — confirming Section 21.2's argument empirically, not just theoretically.

Chapter 22: World Models and Simulation via Diffusion

22.1 Generating Plausible Futures

Beyond generating actions, diffusion models are increasingly used to generate **entire predicted future observations** — e.g. "if the robot takes this action, what will the camera see next?" This extends the World Model idea introduced in the VAE book (its Chapter 19): instead of a VAE-only latent dynamics model, a diffusion model can generate much sharper, more detailed predicted future frames, useful for planning (simulate several candidate action sequences internally, and pick the one whose predicted future looks best) without needing to execute anything in the real world first.

22.2 Named Examples Worth Knowing

- **GAIA-1** (Wayve, 2023) — a generative world model for autonomous driving, trained on real driving video, that can generate realistic and diverse future driving scenarios conditioned on actions, video, and text, used for both simulation and downstream policy training.
- **UniSim / general video-diffusion world models** — large video-diffusion systems trained to predict future frames conditioned on actions across many different embodiments and environments, aiming at a single shared "world simulator" usable across robot types.
- **Diffusion-based dynamics models in Dreamer-style architectures** — replacing the RSSM-style recurrent latent dynamics model (as used in the original Dreamer world model) with a diffusion-based predictor for sharper, more detailed imagined rollouts during planning.

22.3 Synthetic Training Data for Perception

Diffusion models are also widely used to generate synthetic training images for perception systems — e.g. generating extra variations of a scene (different lighting, weather, object placement) to make a vision model (like the YOLO-based or Cognex-style pipelines you've worked with) more robust to conditions that are rare or expensive to physically collect. This directly extends the sim-to-real discussion from the VAE book's robotics chapter, and is one of the most immediately practical, low-risk ways to bring diffusion models into an existing robotics or inspection pipeline.

Chapter 23: Integrating a Diffusion Policy into ROS 2

23.1 Where the Model Sits in the Computation Graph

A trained Diffusion Policy is typically wrapped as a single ROS 2 node that subscribes to sensor topics, runs the denoising loop internally, and publishes the resulting action. Nothing about ROS 2's publish/subscribe or action-server structure needs to change — the diffusion model is simply a more capable function living inside one node.

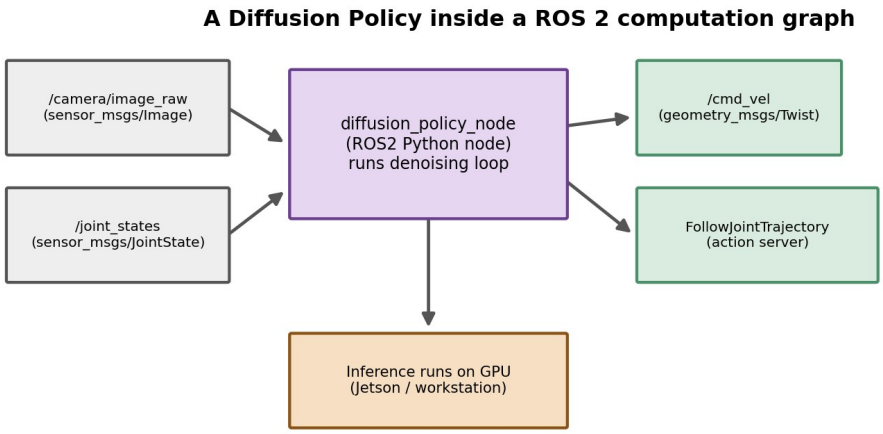


Figure 23.1 — A Diffusion Policy node inside a ROS 2 graph: sensor topics in, an iterative denoising loop inside the node, actions or velocity commands out.

23.2 A Minimal ROS 2 Node Sketch

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, JointState
from geometry_msgs.msg import Twist

class DiffusionPolicyNode(Node):
    def __init__(self):
        super().__init__('diffusion_policy_node')
        self.model = load_trained_diffusion_policy()   # T_o=2, T_p=16, T_a=8
(Sec 21.3)

        self.image_sub = self.create_subscription(
            Image, '/camera/image_raw', self.on_image, 10)
        self.joint_sub = self.create_subscription(
            JointState, '/joint_states', self.on_joints, 10)
        self.cmd_pub = self.create_publisher(Twist, '/cmd_vel', 10)

        self.obs_buffer = []                          # rolling T_o-length
observation window
        self.action_queue = []                         # T_a actions queued for
```

```
execution
        self.timer = self.create_timer(0.1, self.control_loop)   # 10 Hz control
loop

    def on_image(self, msg):
        self.latest_image = preprocess(msg)

    def on_joints(self, msg):
        self.latest_joints = preprocess(msg)

    def control_loop(self):
        if not self.action_queue:                       # queue empty: re-plan
            observation = encode(self.obs_buffer)
            action_sequence = self.model.sample(observation)   # Chapter 13's
loop, conditioned
            self.action_queue = list(action_sequence[:self.model.T_a])
        twist = action_to_twist(self.action_queue.pop(0))
        self.cmd_pub.publish(twist)

def main():
    rclpy.init()
    node = DiffusionPolicyNode()
    rclpy.spin(node)
```

23.3 Quality of Service (QoS) Considerations

Camera and joint-state topics should typically use a `BEST_EFFORT` reliability QoS profile with a small queue depth (e.g. depth=1) rather than the default `RELIABLE` profile — for a real-time control loop, a dropped, slightly-stale frame is far less harmful than blocking on a queued backlog of old frames. The published action/velocity topic, by contrast, is usually kept `RELIABLE` so commands are never silently dropped en route to the robot's driver.

23.4 A Critical Practical Constraint: Inference Speed

Recall from Chapter 14 that full DDPM sampling requires many sequential network passes. For real-time robot control (often needing 10–30 Hz command rates), this is usually far too slow if run naively. In practice, robotics deployments of Diffusion Policy almost always use a **fast sampler** (DDIM-style, typically 5–10 steps, Section 14.2) and run inference on a GPU — an onboard Jetson-class device for a mobile robot/drone, or an offboard workstation streaming commands over your network for a stationary manipulator or AMR setup. Converting the trained U-Net to TensorRT (the same optimization path you've already explored for other perception and control models) is standard practice for meeting real-time latency budgets on embedded hardware.

23.5 A Rough Latency Budget

Stage	Typical latency (Jetson-class GPU, small policy net)
Observation encoding (CNN)	~2-4 ms

Stage	Typical latency (Jetson-class GPU, small policy net)
One U-Net denoising step	~3-6 ms
Full DDIM sampling (8 steps)	~25-50 ms
Action post-processing + publish	<1 ms
Total, per re-plan	~30-55 ms — comfortably inside a 100 ms (10 Hz) control period

23.6 Where This Fits Your Own Work

This pattern — a perception encoder feeding a generative model, wrapped in a ROS 2 node, optimized for real-time inference — is structurally the same pipeline shape as the VLA architecture work and humanoid/TurtleBot3 simulation projects you've been building. A Diffusion Policy node is a drop-in replacement for a classical MPC controller or a directly-regressed policy network at exactly the point in the graph where “observation goes in, action comes out” — the same slot occupied by, for example, an MPC trajectory tracker, just with a fundamentally different (and often more expressive) way of producing the action.

Chapter 24: Where to Go From Here

You now understand diffusion models from the physical ink-drop intuition all the way to their use in generating robot actions. Suggested next steps, in order:

1. **Implement a minimal DDPM from scratch** on a small image dataset (MNIST or CIFAR-10), using the code in Chapter 12, and verify you can reproduce the forward/reverse figures from Chapter 1 yourself.
2. **Implement DDIM sampling** on the same trained model and compare sample quality at 1000 vs. 50 vs. 20 steps.
3. **Read the original DDPM paper** (Ho et al., 2020) and the Score SDE paper (Song et al., 2021) back to back — with this book's Chapters 11 and 17 as a bridge, their equivalence should now be visible directly.
4. **Read the Diffusion Policy paper** (Chi et al., 2023) in full, now that Chapter 21 has given you the vocabulary to follow its derivations without friction.
5. **Try a small Diffusion Policy experiment in simulation** — your existing Gazebo/ROS 2 TurtleBot3 or humanoid setup is a natural, already-available testbed for exactly this.
6. **Read "The Principles of Diffusion Models"** (Lai, Song, Kim, Mitsufuji, Ermon, 2025) — the monograph behind the video you linked — for the fully unified, rigorous treatment of everything in Part V.

> **CLOSING THOUGHT**
>
> Every idea in this book traces back to the same two moves: destroy structure in a simple, known way, and train a network to reverse that destruction. Whether the "structure" is a photograph, a latent code, or a robot's future trajectory, the mathematics you now understand is exactly the same.

PART VII — Exercises and Practice

These problems are designed to be solved with pen and paper first, then checked against the solutions below. They span the whole book, weighted toward the mathematical core in Parts II and III.

Chapter 25: Exercises and Practice Problems

25.1 Conceptual Questions

1. In your own words, explain why the forward process requires no learning but the reverse process does.
2. Why is it valid for the network's output distribution $p_\theta(x_{t-1}|x_t)$ to be assumed Gaussian? What specific result justifies this?
3. Explain the difference between what $q(x_{t-1}|x_t)$ and $q(x_{t-1}|x_t,x_0)$ can tell us, and why only one of them is tractable.
4. A colleague says classifier-free guidance requires training two separate networks. Correct this misunderstanding.
5. Why does DDIM allow skipping timesteps safely, when naively skipping steps in the original DDPM sampler would not work as well?
6. Explain, without equations, why Diffusion Policy tends to outperform a directly-regressed (MSE-trained) policy network on tasks with multiple valid solutions.

25.2 Computation Practice

1. Given $\alpha_1 = 0.95$ and $\alpha_2 = 0.90$, compute $\bar{\alpha}_2$.
2. Using $\bar{\alpha}_2$ from the previous question, $x_0 = 1.0$, and $\varepsilon = -0.8$, compute x_2 using the closed-form shortcut.
3. A noise schedule has $\bar{\alpha}_{500} = 0.5$ at the halfway point. Roughly what fraction of the original signal's standard deviation remains at that timestep? (Hint: look at the coefficient on x_0.)
4. For classifier-free guidance with $w=1$, what does the final formula simplify to? What does this tell you about $w=1$ as a baseline?

25.3 Design Questions

1. You are deploying a Diffusion Policy on a mobile robot with a 20 Hz control loop requirement and only a small embedded GPU. List two concrete changes from Part IV you would apply, and why.
2. You need a robot to generate future camera-frame predictions for internal planning, several seconds into the future. Would you reach for a plain VAE-based world model or a diffusion-based one? Justify using Chapter 22.
3. You're training a text-to-image model and notice generated images look over-saturated and cartoonish at high guidance weights. Which chapter's fix would you apply first?

25.4 Solutions

Conceptual Questions

- **Q1** — The forward process is a fixed, simple, hand-designed Gaussian formula (Chapter 4) — there is nothing to learn because we choose it ourselves. The reverse process requires inverting that formula, which depends on the entire unknown data distribution (Section 8.1), so it must be approximated by a trained network.
- **Q2** — Section 8.2's derivation shows the TRUE posterior q(x_(t-1)|x_t,x_0), conditioned on x_0, is exactly Gaussian. This is a rigorous mathematical result (given the Gaussian forward process), not just a convenient approximation — so it is justified, not merely assumed for convenience, that the network's approximation should also be Gaussian-shaped.
- **Q3** — q(x_(t-1)|x_t) alone requires the intractable marginal q(x_(t-1)) (Section 8.1). q(x_(t-1)|x_t,x_0), which additionally conditions on the clean data point x_0, has a closed-form Gaussian solution (Section 8.2) because x_0 is known during training, removing the need for that troublesome marginal.
- **Q4** — Classifier-free guidance uses a SINGLE network trained to handle both conditional and unconditional prediction (by randomly dropping the condition during training), not two separate networks (Section 15.2). Classifier GUIDANCE (no “free”), by contrast, does require a second, separate classifier network (Section 15.1) — the colleague may be thinking of that older, different technique.
- **Q5** — DDIM's update rule (Section 14.2) is derived from a non-Markovian process sharing the same marginals as DDPM, which remains valid at any subset of timesteps by construction. The original DDPM sampler's derivation assumes consecutive, adjacent timesteps, so skipping steps there introduces error the formula was never designed to tolerate.
- **Q6** — A directly-regressed policy trained with MSE loss is forced to predict a single action, and when multiple valid actions exist (Section 21.2), MSE training pushes it toward their average — which is often not itself a valid action. A diffusion policy samples one full, self-consistent solution from a learned distribution instead of averaging across the possibilities.

Computation Practice

- **Q1** — $\bar{\alpha}_2 = 0.95 \times 0.90 = \mathbf{0.855}$
- **Q2** — $x_2 = \sqrt{0.855}\times 1.0 + \sqrt{(1-0.855)}\times(-0.8) = 0.9247 - 0.8\times 0.3808 = 0.9247 - 0.3046 = \mathbf{0.6201}$
- **Q3** — The signal coefficient is $\sqrt{\bar{\alpha}_{500}} = \sqrt{0.5} \approx \mathbf{0.707}$, so roughly 71% of the original signal's scale remains — notably more than half, even though the VARIANCE contribution ($\bar{\alpha}_{500}$ itself) is exactly half.
- **Q4** — With w=1, $\varepsilon_{final} = \varepsilon_{uncond} + 1\times(\varepsilon_{cond} - \varepsilon_{uncond}) = \boldsymbol{\varepsilon_{cond}}$ exactly — plain conditional generation with no extra guidance boost. This confirms w=1 is the sensible, guidance-free baseline case.

Design Questions (Sample Reasoning)

- **Q1** — (1) Use a DDIM-style fast sampler with very few steps (Section 14.2), and (2) convert the U-Net to TensorRT and/or reduce model size, following the latency budget approach in Section 23.5, to fit inside a ~50ms control period.
- **Q2** — A diffusion-based world model (Section 22.1–22.2), since it produces sharper, more detailed predicted frames than a VAE-only latent dynamics model — valuable when the predictions themselves need to be visually reliable for planning several seconds out, at the cost of more expensive inference than a pure VAE rollout.
- **Q3** — Guidance rescaling (Section 15.4), which directly targets the over-saturation artifact caused by high guidance weights, before resorting to simply lowering w (which would also reduce prompt adherence).

Appendix A: Notation Reference

Symbol	Meaning
x_0	The original, clean data point (e.g. a real image)
x_t	The data after t steps of forward noising
x_T	The fully-noised endpoint, ≈ pure Gaussian noise
T	Total number of diffusion steps (e.g. 1000)
β_t	Noise added at step t (the noise schedule)
α_t	1 - β_t
ᾱ_t (alpha-bar)	Cumulative product α_1·α_2·…·α_t
q(x_t|x_(t-1))	The forward (noising) process — fixed, not learned
p_θ(x_(t-1)|x_t)	The reverse (denoising) process — learned
μ̃_t, β̃_t	The exact closed-form posterior mean/variance given x_0 (Chapter 8)
ε	Sampled Gaussian noise, ε ~ N(0,1)
ε_θ(x_t, t)	The U-Net's predicted noise
∇log p(x_t)	The score function (gradient of log-density)
y	Conditioning information (e.g. a text prompt or robot observation)
w	Classifier-free guidance weight
σ_t	DDIM's free stochasticity parameter (0 = fully deterministic)
f(x,t), g(t)	SDE drift and diffusion coefficients (Chapter 17)
T_o, T_p, T_a	Diffusion Policy's observation / prediction / execution horizons (Chapter 21)

Appendix B: Glossary

- **Diffusion** — the physical process of a concentrated substance spreading into a uniform, structureless distribution over time.
- **Forward process** — the fixed, mathematically known procedure of gradually adding noise to data.
- **Reverse process** — the learned procedure of gradually removing noise, used to generate new samples.
- **Noise schedule** — the sequence of β_t values controlling how much noise is added at each step; linear and cosine are the two most common designs.
- **Markov chain** — a sequence of steps where each step depends only on the immediately preceding one.
- **U-Net** — an encoder-decoder convolutional architecture with skip connections, used as the standard noise-prediction network.
- **Score function** — the gradient of the log probability density, pointing toward higher-density regions of data.
- **Classifier-free guidance** — a technique for steering generation toward a condition (like a prompt) without a separate classifier network.
- **DDPM** — Denoising Diffusion Probabilistic Model; the original, full-step diffusion formulation.
- **DDIM** — Denoising Diffusion Implicit Models; a faster, optionally-deterministic sampling procedure using fewer steps.
- **Latent Diffusion** — running the diffusion process inside a compressed VAE latent space rather than on raw pixels.
- **Probability Flow ODE** — the deterministic differential equation whose trajectories match the diffusion process's marginal distributions exactly.
- **Flow Matching** — a generalized formulation that learns a velocity field along a path from noise to data.
- **Consistency Model** — a model trained to map any point on a noise-to-data trajectory directly to the same final sample, enabling one-step generation.
- **Diffusion Policy** — applying the diffusion generative framework to robot action sequences instead of images.
- **Receding-horizon control** — replanning a short future window of actions repeatedly, executing only the first portion before replanning again.
- **FID** — Fréchet Inception Distance; the standard metric for comparing generated and real image distributions.

Appendix C: Formula Cheat Sheet

Name	Formula
Single forward step	$x_t = \sqrt{\alpha_t} \cdot x_{t-1} + \sqrt{1-\alpha_t} \cdot \varepsilon$
Closed-form jump to step t	$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{1-\bar{\alpha}_t} \cdot \varepsilon$
Cumulative signal	$\bar{\alpha}_t = \alpha_1 \cdot \alpha_2 \cdot \ldots \cdot \alpha_t$
Exact posterior mean (given x_0)	$\tilde{\mu}_t = (\sqrt{\bar{\alpha}_{t-1}}\beta_t/(1-\bar{\alpha}_t))x_0 + (\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1}))/(1-\bar{\alpha}_t))x_t$
Exact posterior variance	$\tilde{\beta}_t = ((1-\bar{\alpha}_{t-1}))/(1-\bar{\alpha}_t)) \cdot \beta_t$
x_0 estimate from predicted noise	$\hat{x}_0 = (x_t - \sqrt{1-\bar{\alpha}_t} \cdot \varepsilon_\theta) / \sqrt{\bar{\alpha}_t}$
Simplified training loss	$L = E[\ \lvert\lvert \varepsilon - \varepsilon_\theta(x_t,t) \rvert\rvert^2\ ]$
DDIM update (general σ_t)	$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \cdot \hat{x}_0 + \sqrt{1-\bar{\alpha}_{t-1}-\sigma_t^2} \cdot \varepsilon_\theta + \sigma_t \cdot z$
Classifier-free guidance	$\varepsilon_{final} = \varepsilon_{uncond} + w \cdot (\varepsilon_{cond} - \varepsilon_{uncond})$
Score ↔ noise-prediction link	$\nabla \log p(x_t) \approx -\varepsilon_\theta(x_t,t) / \sqrt{1-\bar{\alpha}_t}$
Forward SDE	$dx = f(x,t)dt + g(t)dw$
Probability Flow ODE	$dx = [f(x,t) - \frac{1}{2}g(t)^2 \nabla \log p_t(x)]dt$
Conditional Flow Matching loss	$L = E[\ \lvert\lvert (x_1' - x_0') - v_\theta(x_\tau,\tau) \rvert\rvert^2\ ]$

Appendix D: Debugging Checklist for Diffusion Model Code

Symptom: Loss is NaN almost immediately

- Check ᾱ_t never reaches exactly 0 or 1 numerically (clip the schedule slightly if using float16).
- Verify the noise schedule tensor (α_bars) is indexed correctly per-sample when t is a batch of different random timesteps, not a single scalar.
- Confirm images are normalized to the range the model expects (commonly [-1, 1]) before computing x_t.

Symptom: Samples are blurry or low-detail even after many training steps

- Check the noise schedule (Section 5.2) — a poorly-tuned linear schedule can waste capacity; try a cosine schedule.
- Confirm you're sampling with enough steps for the chosen sampler (DDPM needs closer to the full T; DDIM can use far fewer but too few will still blur fine detail).
- Verify EMA weights (Section 12.3) are being used for sampling/evaluation, not the raw, noisier training weights.

Symptom: Generated samples don't match the prompt at all

- Confirm the conditioning dropout rate used during training (Section 15.2) isn't set so high that the model rarely learned the conditional pathway.
- Check the guidance weight w isn't accidentally left at 1 (no boost) when stronger prompt adherence is expected.
- Verify the text/conditioning encoder's output is actually being passed into the cross-attention layers, not silently dropped somewhere in the data pipeline.

Symptom: A Diffusion Policy is too slow for the robot's control loop

- Switch to a DDIM-style sampler with a small step count (Section 14.2, Section 23.4) rather than full DDPM sampling.
- Profile observation encoding separately from the denoising loop — a heavy vision backbone can dominate latency as much as the diffusion steps themselves.
- Confirm T_a (execution horizon, Section 21.3) is set high enough that re-planning isn't happening far more often than necessary.

Appendix E: Index of Figures

Figure	Topic
1.1	The ink-drop physical analogy
1.2 / 1.3	Forward and reverse diffusion process
2.1	The score function as a vector field
4.1	A single forward noising step
5.1	The noise schedule (β_t and ᾱ_t)
7.1	The U-Net denoiser architecture
8.1	The training objective, end to end
9.1	The sampling loop
10.1	DDPM vs. DDIM step comparison
11.1	Classifier-free guidance
12.1	Latent Diffusion pipeline
13.1	Diffusion Policy for robot control
14.1	A Diffusion Policy inside a ROS 2 graph
15.1	Three unified perspectives on diffusion

Appendix F: Further Reading

- Ho, J., Jain, A., & Abbeel, P. (2020). Denoising Diffusion Probabilistic Models. — the DDPM paper; the source of Chapters 4, 5, 11, and 12.
- Nichol, A., & Dhariwal, P. (2021). Improved Denoising Diffusion Probabilistic Models. — the cosine schedule, source of Section 5.2.
- Song, J., Meng, C., & Ermon, S. (2021). Denoising Diffusion Implicit Models. — the DDIM paper behind Chapter 14.
- Dhariwal, P., & Nichol, A. (2021). Diffusion Models Beat GANs on Image Synthesis. — classifier guidance, source of Section 15.1.
- Ho, J., & Salimans, T. (2022). Classifier-Free Diffusion Guidance. — source of Section 15.2.
- Song, Y., et al. (2021). Score-Based Generative Modeling through Stochastic Differential Equations. — the continuous-time / score-based view in Chapter 17.
- Rombach, R., et al. (2022). High-Resolution Image Synthesis with Latent Diffusion Models. — the Stable Diffusion paper, source of Chapter 16.
- Lipman, Y., et al. (2023). Flow Matching for Generative Modeling. — source of Chapter 18.
- Song, Y., et al. (2023). Consistency Models. — source of Chapter 19.
- Chi, C., et al. (2023). Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. — source of Chapter 21, the robotics connection.
- Wayve (2023). GAIA-1: A Generative World Model for Autonomous Driving. — source of Section 22.2.
- Lai, C-H., Song, Y., Kim, D., Mitsufuji, Y., & Ermon, S. (2025). The Principles of Diffusion Models: From Origins to Advances. — the unified monograph behind the video you linked; the deepest and most rigorous treatment of everything in Part V.

End of textbook.